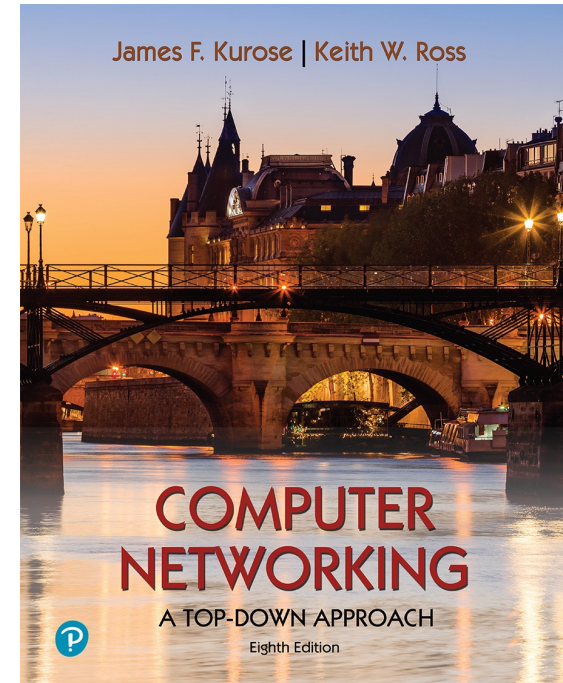


Application networking & programming

All material copyright 1996-2020
J.F Kurose and K.W. Ross, All Rights Reserved
A nice tutorial <https://beej.us/guide/bgnet/html/#what-is-a-socket>



Computer Networking: A Top-Down Approach

8th edition
Jim Kurose, Keith Ross
Pearson, 2020

Class objectives

- **Learn** conceptual *and* implementation aspects of application-layer protocols
 - transport-layer service models
 - client-server paradigm
 - peer-to-peer paradigm
- **Learn** about protocols by examining popular application-layer protocols and infrastructure
 - Web/HTTP
 - DNS
- **Be able to** program network applications
 - socket API

Class organization

- Lectures (2h)
 - Application layer
 - UNIX Network programming API – Socket API
- Labs (8h)
 - C tips and reminders
 - Socket API and network primitives
 - Exercises
- Project assignment (10h40 in class + 10h40 homework)
 - Online messaging application
 - *Non-rattrapable*
 - Deliverables: code + report on Github

Who is who

- Instructor: Dr. Joachim Bruneau-Queyreix
 - Researcher in distributed systems and security @ LaBRI
 - Professor @ Enseirb-matmeca
 - jbruneauqueyreix@enseirb-matmeca.fr

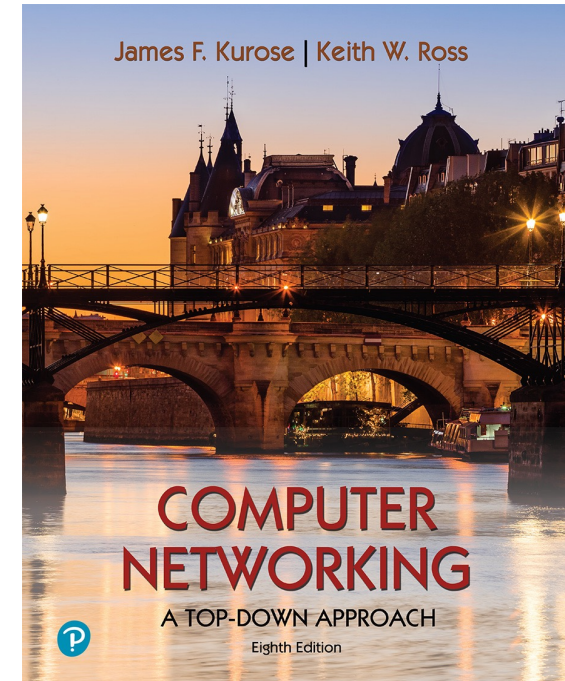
- Teaching assistant: Augusta Mukam
 - Researcher in distributed system @ LaBRI
 - Ph.D candidate
 - amukam@u-Bordeaux.fr

Contacts

- General questions
 - jbruneauqueyreix@enseirb-matmeca.fr
- Questions on the lab sessions
 - jbruneauqueyreix@enseirb-matmeca.fr
- Questions on the project
 - amukam@u-Bordeaux.fr
 - Always add jbruneauqueyreix@enseirb-matmeca.fr in cc

Lecture overview

- Principles of network applications
- Web and HTTP
- The Domain Name System DNS
- P2P applications
- Socket programming with UDP and TCP



Computer Networking: A Top-Down Approach

8th edition
Jim Kurose, Keith Ross
Pearson, 2020

Some network apps

- social networking
- Web
- text messaging
- e-mail
- multi-user network games
- streaming stored video (YouTube, Hulu, Netflix)
- P2P file sharing
- voice over IP (e.g., Skype)
- real-time video conferencing (e.g., Zoom)
- Internet search
- remote login
- ...

Q: *your* favorites?

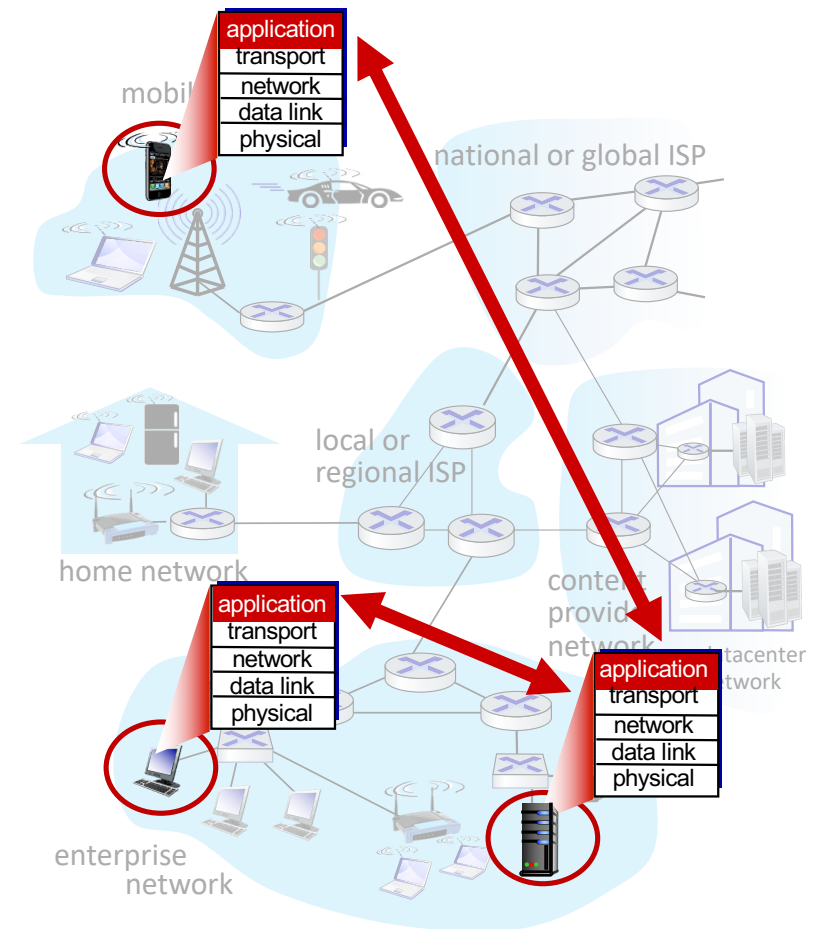
Creating a network app

write programs that:

- run on (different) end systems
- communicate over network
- e.g., web server software communicates with browser software

no need to write software for network-core devices

- network-core devices do not run user applications
- applications on end systems allows for rapid app development, propagation



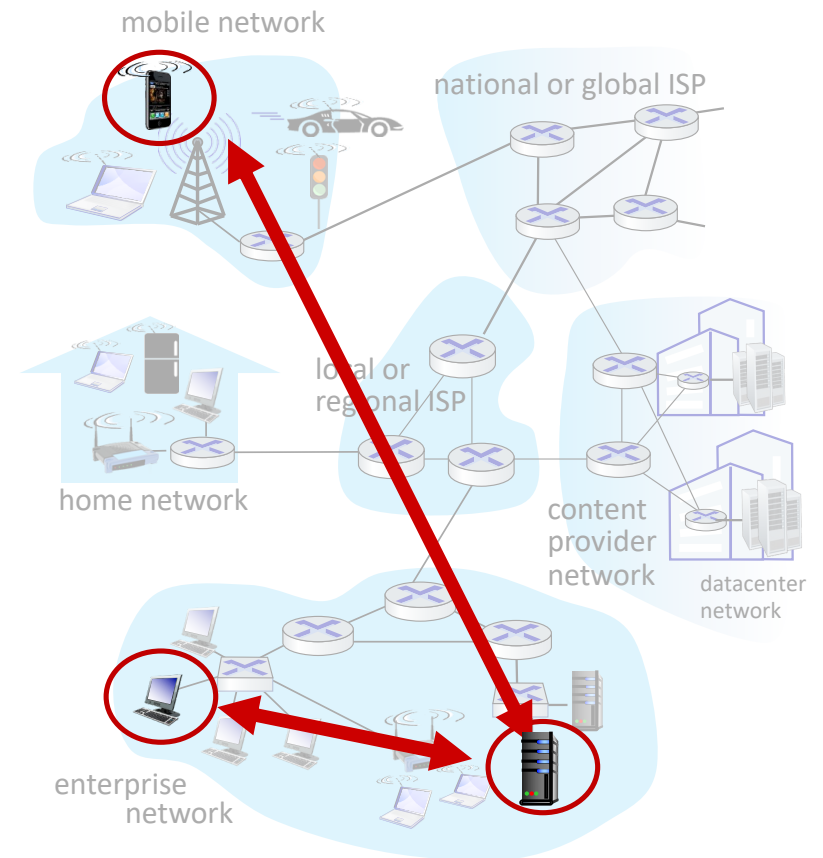
Client-server paradigm

server:

- always-on host
- permanent IP address and port#
- often in data centers, for scaling

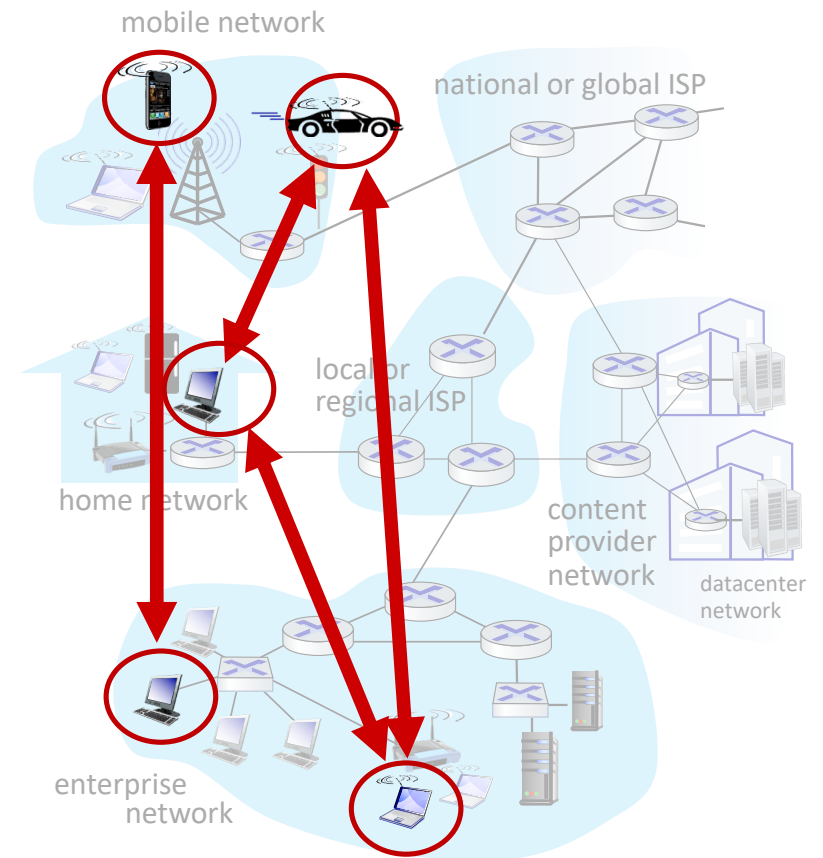
clients:

- contact, communicate with server
- may be intermittently connected
- may have dynamic IP addresses
- do *not* communicate directly with each other
- examples: HTTP, IMAP, FTP



Peer-peer architecture

- *no* always-on server
- arbitrary end systems directly communicate
- peers request service from other peers, provide service in return to other peers
 - *self scalability* – new peers bring new service capacity, as well as new service demands
- peers are intermittently connected and change IP addresses
 - complex management
- example: P2P file sharing



Processes communicating

process: program running within a host

- within same host, two processes communicate using *inter-process communication* (defined by OS)
- processes in different hosts communicate by exchanging *messages*

clients, servers

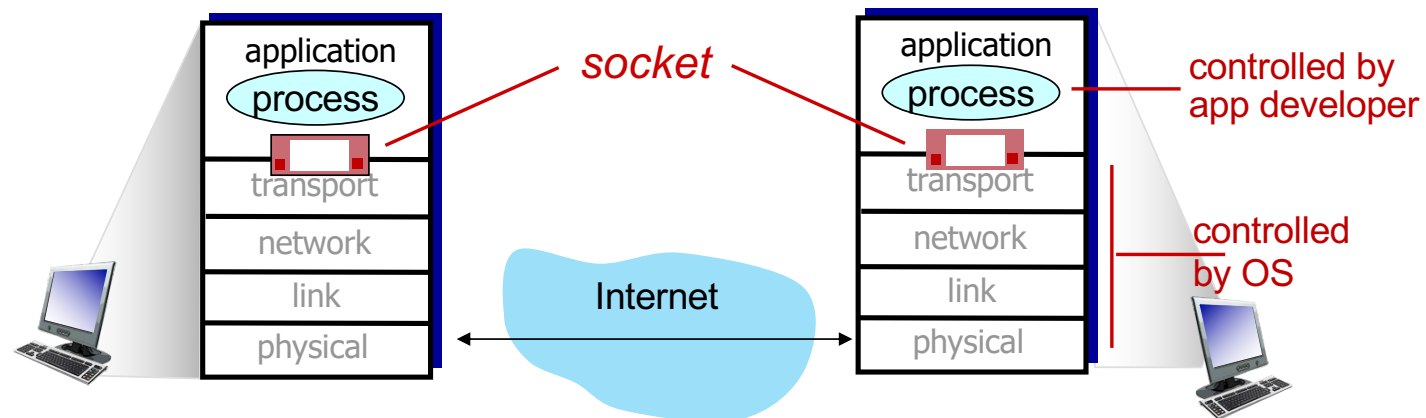
client process: process that initiates communication

server process: process that waits to be contacted

- note: applications with P2P architectures have client processes & server processes

Sockets

- process sends/receives messages to/from its **socket**
- socket analogous to door
 - sending process shoves message out door
 - sending process relies on transport infrastructure on other side of door to deliver message to socket at receiving process
 - two sockets involved: one on each side



Addressing processes

- to receive messages, process must have *identifier*
- host device has unique 32-bit IP address
- Q: does IP address of host on which process runs suffice for identifying the process?
- A: no, *many* processes can be running on same host
- *identifier* includes both **IP address** and **port numbers** associated with process on host.
- example port numbers:
 - HTTP server: 80
 - mail server: 25
- to send HTTP message to gaia.cs.umass.edu web server:
 - **IP address:** 128.119.245.12
 - **port number:** 80
- more shortly...

An application-layer protocol defines:

- **types of messages exchanged**,
 - e.g., request, response
- **message syntax**:
 - what fields in messages & how fields are delineated
- **message semantics**
 - meaning of information in fields
- **rules** for when and how processes send & respond to messages

open protocols:

- defined in RFCs, everyone has access to protocol definition
- allows for interoperability
- e.g., HTTP, SMTP

proprietary protocols:

- e.g., Skype, Zoom

What transport service does an app need?

data integrity

- some apps (e.g., file transfer, web transactions) require 100% reliable data transfer
- other apps (e.g., audio) can tolerate some loss

timing

- some apps (e.g., Internet telephony, interactive games) require low delay to be “effective”

throughput

- some apps (e.g., multimedia) require minimum amount of throughput to be “effective”
- other apps (“elastic apps”) make use of whatever throughput they get

security

- encryption, data integrity, ...

Transport service requirements: common apps

application	data loss	throughput	time sensitive?
file transfer/download	no loss	elastic	no
e-mail	no loss	elastic	no
Web documents	no loss	elastic	no
real-time audio/video	loss-tolerant	audio: 5Kbps-1Mbps video:10Kbps-5Mbps	yes, 10's msec
streaming audio/video	loss-tolerant	same as above	yes, few secs
interactive games	loss-tolerant	Kbps+	yes, 10's msec
text messaging	no loss	elastic	yes and no

Internet transport protocols services

TCP service:

- *reliable transport* between sending and receiving process
- *flow control*: sender won't overwhelm receiver
- *congestion control*: throttle sender when network overloaded
- *connection-oriented*: setup required between client and server processes
- *does not provide*: timing, minimum throughput guarantee, security

UDP service:

- *unreliable data transfer* between sending and receiving process
- *does not provide*: reliability, flow control, congestion control, timing, throughput guarantee, security, or connection setup.

Q: why bother? *Why* is there a UDP?

Internet applications, and transport protocols

application	application layer protocol	transport protocol
file transfer/download	FTP [RFC 959]	TCP
e-mail	SMTP [RFC 5321]	TCP
Web documents	HTTP 1.1 [RFC 7320]	TCP
Internet telephony	SIP [RFC 3261], RTP [RFC 3550], or proprietary	TCP or UDP
streaming audio/video	HTTP [RFC 7320], DASH	TCP
interactive games	WOW, FPS (proprietary)	UDP or TCP

Securing TCP

Vanilla TCP & UDP sockets:

- no encryption
- cleartext passwords sent into socket traverse Internet in cleartext (!)

Transport Layer Security (TLS)

- provides encrypted TCP connections
- data integrity
- end-point authentication

TLS implemented in application layer

- apps use TLS libraries, that use TCP in turn
- cleartext sent into “socket” traverse Internet *encrypted*

Lecture overview

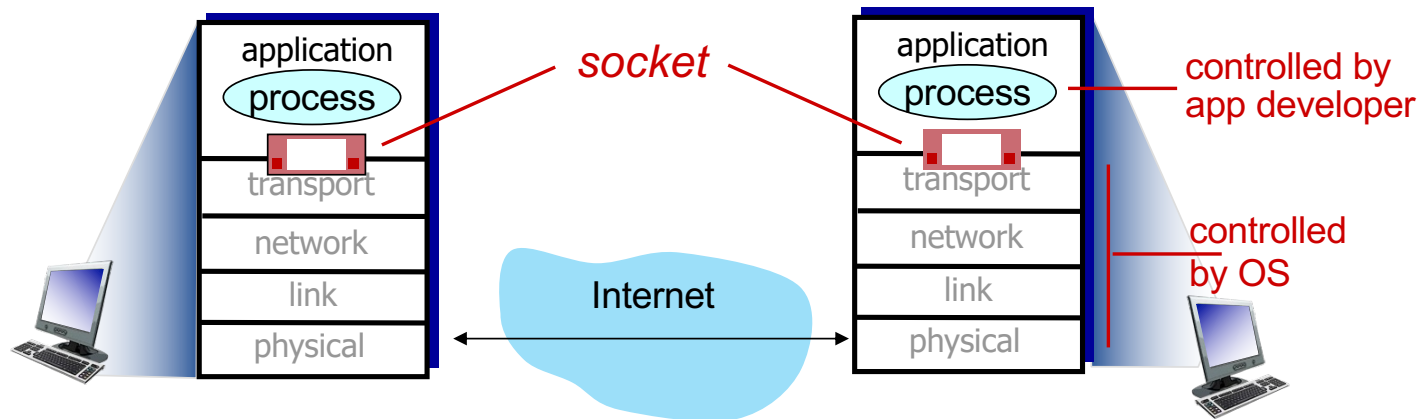
- Principles of network applications
- Web and HTTP
- The Domain Name System DNS
- P2P applications
- socket programming with UDP and TCP



Socket programming

goal: learn how to build client/server applications that communicate using sockets

socket: door between application process and end-end-transport protocol



Sockets

- Communication interface introduced by Unix systems for network communications
- Set of primitives
- Generic
 - Adapts to different communication needs
- Independent of particular protocols and networks
- Does not necessarily rely on a given network
 - Example: local communication(loopback), Unix domain

Socket programming

Two socket types for two transport services:

- *UDP*: unreliable datagram
- *TCP*: reliable, byte stream-oriented

Application Example:

1. client reads a line of characters (data) from its keyboard and sends data to server
2. server receives the data and converts characters to uppercase
3. server sends modified data to client
4. client receives modified data and displays line on its screen

Socket libraries

- Among others
 - `#include <sys/types.h>`
 - `#include <sys/socket.h>`
 - `#include <netdb.h>`

- Check missing libraries from the man pages

Socket creation

int socket(int domain, int type, int protocol);

- Parameters:
 - Domain: protocol family
 - Type: protocol type
 - Protocol: specific to protocol versions => leave it to 0
- Result:
 - Socket file descriptor (int) or -1 if failure and errno is set to error value
- Example
 - `Int fd = socket(AF_INET,SOCK_STREAM,0);`

Frequent errors

- Missing libraries for specific functions => look up the appropriate `#include` to set from the man pages
- Port numbers are not in the correct order (network or host)
- You're trying to connect a client to dead server
- You're not properly compiling your code
- You're not reading from/writing to sockets correctly

Socket destruction

int close(int s);

- Parameters:
 - s: socket file descriptor
- Result:
 - 0 if success, -1 if failure with errno set accordingly
- N.B.:
 - Prevents any read/write operations once closed

Tuning and reading socket options

*int getsockopt(int s, int level, int optname, void *optval, int *optlen)*

*int setsockopt(int s, int level, int optname, const void *optval, int optlen)*

- Parameters:
 - s: socket file descriptor
 - Level: option level to be handled (socket or protocol)
 - Optname: option name to be handled (vector)
- Result:
 - 0 if success, -1 if failure with errno set accordingly
- Useful when re-using the same port number after process failure

Socket programming with TCP

Client must contact server

- server process must first be running
- server must have created socket (door) that welcomes client's contact

Client contacts server by:

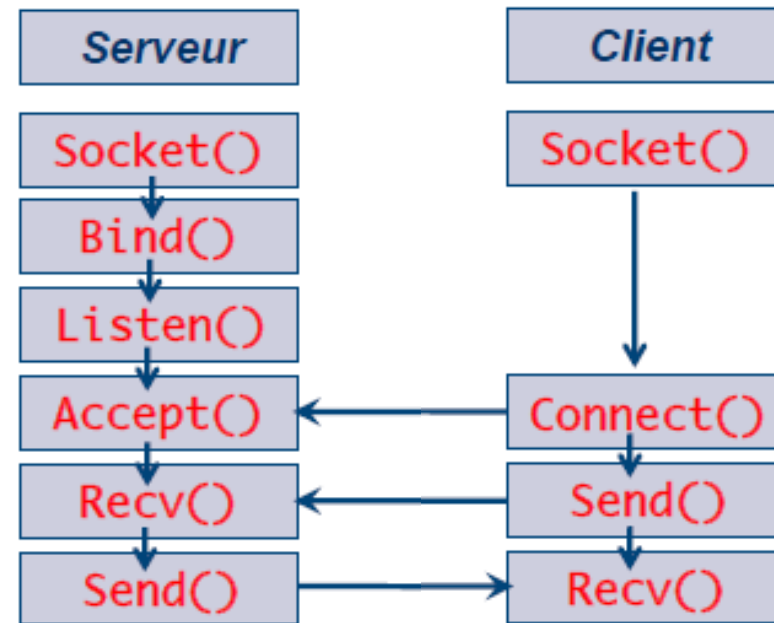
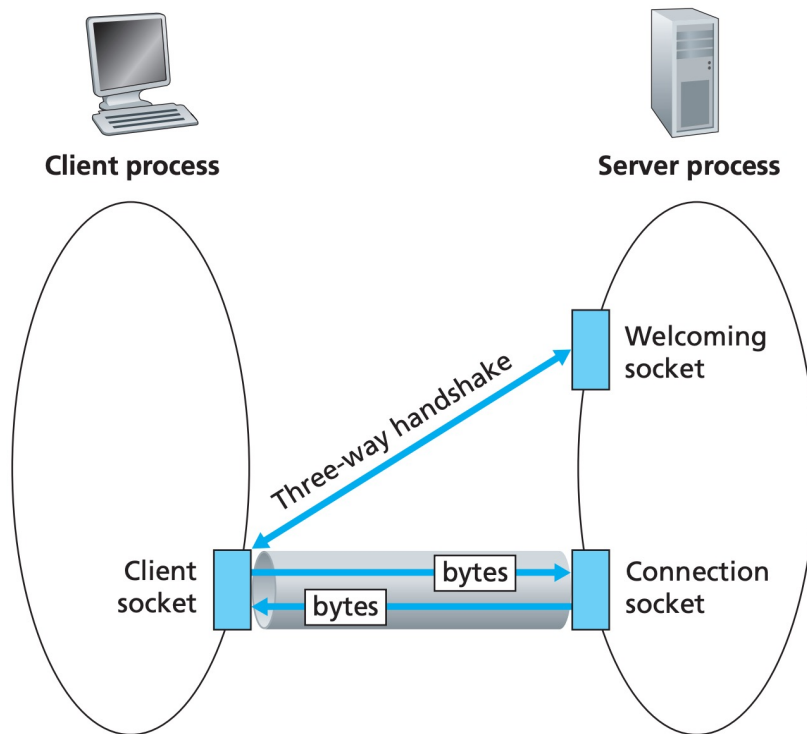
- Creating TCP socket, specifying IP address, port number of server process
- *when client creates socket*: client TCP establishes connection to server TCP

- when contacted by client, *server TCP creates new socket* to communicate with that particular client
 - allows server to talk with multiple clients
 - *source* port numbers used to distinguish clients
 - Three strategies to handle the new sockets:
 - Sequential, processes, threads, I/O multiplexing

Application viewpoint

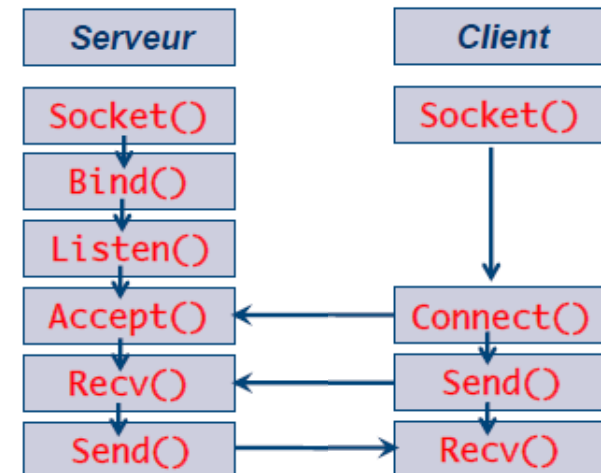
TCP provides reliable, in-order byte-stream transfer ("pipe") between client and server processes

Socket programming with TCP



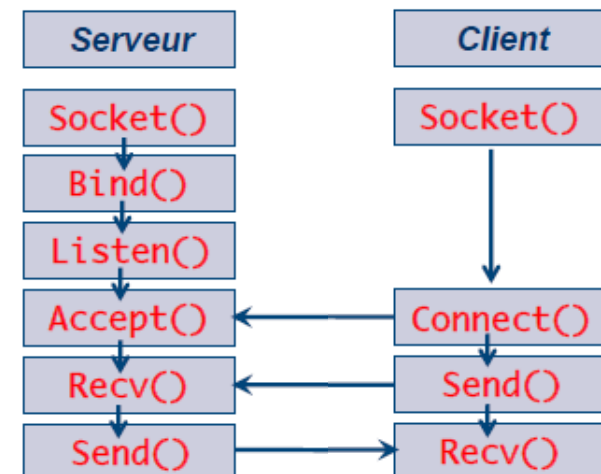
Socket programming with TCP

- **Côté client** ⇔ demandeur de la connexion ⇔ ACTIF
 - socket()
 - Permet de créer une socket
 - connect()
 - Permet de se connecter à un couple (adresse IP, port)
 - read(), write(), send(), recv()
 - **Pour lire et écrire dans la socket**
 - close()
 - Pour fermer la socket



Socket programming with TCP

- **Côté serveur** ⇔ en attente de connexion ⇔ **PASSIF**
 - `socket()`
 - Pour créer une socket
 - `bind()`
 - Pour associer une adresse à la socket
 - `listen()`
 - Pour se mettre à l'écoute des connexions
 - `accept()`
 - Pour accepter une connexion
 - `read()`, `write()`, `send()`, `recv()`
 - Pour lire et écrire sur la socket
 - `close()`
 - Pour fermer la socket



Socket binding

```
int bind(int s, const struct sockaddr *name, int namelen);
```

- Parameters:
 - S: socket descriptor
 - Name: address to bind socket s on
 - Namelen: length of name
- Result:
 - 0 if success, -1 if failure with errno set accordingly
- N.B.:
 - INADDR_ANY can be used in *name* to specify any address on the local host
 - Port number should be greater than 1024 for safe usage of your services

General address structure

- La structure générique pour les adresses :

```
struct sockaddr {  
    u_short sa_family;  
    char sa_data[14];  
};
```

sa_family (AF_INET/AF_INET6 pour IPv4/6)

sa_data: (IP + numéro de port) => **Pas pratique à manipuler**

Struct sockaddr_in

- *Structure définie dans <netinet/in.h>*
 - sin_family représente le type de famille AF_INET
 - sin_port représente le port à contacter
 - sin_addr représente l'adresse IP de l'hôte
 - sin_zero[8] contient uniquement des zéros (permet de se caler sur la structure sock_addr)

```
struct sockaddr_in {  
    short sin_family; /* AF_INET */  
    u_short sin_port; /* le numéro de port */  
    struct in_addr sin_addr; /* l'adresse Internet */  
    char sin_zero[8]; /* un champ de 8 zéros */  
};  
  
struct in_addr {  
    uint32_t s_addr; /* 4 octets */  
};
```

- Pour accéder au champ contenant l'adresse IPv4 :

```
struct sockaddr_in mysock;  
mysock.sin_addr.s_addr;
```

Example: Socket binding with *struct sockaddr_in*

```
#include <sys/types.h>
#include <sys/socket.h>

int bind(int s, const struct sockaddr *adr, int adrlen);
```

- Côté serveur
 - Besoin de spécifier #numéro de port lors de l'appel *bind()*,
 - ⇒ pour que d'autres processus puissent contacter le serveur sur la socket ouverte
- Côté client
 - Le numéro de port peut ne pas être spécifié (port=0).
 - le système choisira lui même un port
- Pour connaître l'@IP source on utilisera INADDR_ANY comme paramètre
- Nécessaire de caster un *struct sockaddr** en *struct sockaddr_in**

Example

- *Création d'une adresse pour un client*

```
SOCKADDR_IN sin;  
sin.sin_addr.s_addr = htonl(INADDR_ANY);  
sin.sin_family = AF_INET;  
sin.sin_port = 0; //Port alloué dynamiquement
```

- *Création d'une adresse pour un serveur qui écoute sur le port 33000*

```
SOCKADDR_IN sin;  
sin.sin_addr.s_addr = htonl(INADDR_ANY);  
sin.sin_family = AF_INET;  
■ sin.sin_port = htons(33000); //Port port 33000
```

```
bind(sock, (SOCKADDR*)&sin, sizeof(sin))
```

Conversion host-to-network

```
char * inet_ntoa(const struct in_addr in);  
in_addr_t inet_addr(const char *cp);  
int inet_aton (const char *cp, struct in_addr *inp);
```

- `inet_ntoa()` : Convertit la structure binaire `in` en chaîne de caractères (notation décimale pointée)
- `inet_addr()` (obsolète): Convertit la chaîne `cp` en binaire et si l'adresse `cp` est invalide `INADDR_NONE` est renvoyée
- `inet_aton()`: Idem que `inet_addr` mais plus récent

```
struct sockaddr_in myaddr;  
char *some_addr;
```

```
inet_aton("10.0.0.1", &myaddr.sin_addr);  
some_addr = inet_ntoa(myaddr.sin_addr);  
printf("%s\n", some_addr);
```

```
// and this call is the same as the inet_aton() call, above:  
myaddr.sin_addr.s_addr = inet_addr("10.0.0.1");
```

Integer conversion

```
unsigned long htonl(unsigned long hostlong);  
unsigned short htons(unsigned short hostshort);  
unsigned long ntohl(unsigned long netlong);  
unsigned short ntohs(unsigned short netshort);
```

- Deux méthodes pour enregistrer les entiers
 - Little endian et Big endian
- Les ordinateurs enregistrent dans l'ordre défini par leur processeur (Host Byte Order)
- Exemple:
 - Intel x86, *Host Byte Order* est *Little-Endian*
- L'ordre dans lequel les octets doivent être écrits sur le réseau est Big endian

Reading a socket address from a socket descriptor

```
int getsockname(int s, struct sockaddr *name, int *namelen);
```

- Parameters:
 - S: socket descriptor
 - Namelen: pointer to int that include the size of *name
- Output parameters
 - *name*: pointer to created struct sockaddr that will be filled by the function
- Result:
 - 0 if success, -1 if failure with errno set accordingly

Listening for incoming connections

int listen(int s, int backlog);

- Parameters:
 - S: socket descriptor
 - backlog: incoming connection queue length
- Result:
 - 0 if success, -1 if failure with errno set accordingly

Accepting incoming connections

*int accept(int s, struct sockaddr *addr, int *addrlen);*

- Input parameters:
 - S: socket descriptor
- Parameters modified after function call:
 - addr: struct sockaddr_in filled by the function with the address of the incoming connection
 - Addrlen: length of addr
- Result:
 - New socket file descriptor upon success, -1 if failure with errno set accordingly

Connecting to a remote socket

*int connect(int s, const struct sockaddr *name, int namelen)*

- Input parameters:
 - S: socket descriptor
 - *name: struct sockaddr_in including the remote server's address
 - namelen: length of namelen*
- Result:
 - 0 success, -1 if failure with errno set accordingly

Reading from a socket

*ssize_t read(int s, void *buf, size_t nbytes);*

- Input parameters
 - s: socket descriptor
 - Buf: pointer to an memory buffer where data will be stored
 - nbytes: maximum number of bytes to read from the socket
- Result:
 - Number of bytes read, -1 if failure with errno set accordingly

Writing to socket

*ssize_t write(int s, const void *buf, size_t nbytes);*

- Input parameters
 - s: socket descriptor
 - Buf: pointer to a memory buffer of data to write to the socket
 - nbytes: maximum number of bytes to write to the socket
- Result:
 - Number of bytes written, -1 if failure with errno set accordingly

TCP server example

```
struct sockaddr_in saddr_in;  
int sock;
```

Déclaration des variables

```
sock = socket(PF_INET, SOCK_STREAM, 0);  
if ( sock == -1 ) {  
    perror( "socket" ); exit( EXIT_FAILURE );  
}
```

Création de la socket

```
memset(&saddr_in, 0, sizeof (saddr_in));  
saddr_in.sin_family = AF_INET;  
saddr_in.sin_port = htons( port );  
saddr_in.sin_addr.s_addr = INADDR_ANY;
```

Initialisation de l'adresse

```
if ( bind( sock, (struct sockaddr *) &saddr_in, sizeof(saddr_in) ) == -1 ){  
    perror( "bind" );  
    exit( EXIT_FAILURE );  
}
```

Attachement de l'adresse à la socket

```
if ( listen( sock, SOMAXCONN ) == -1 ) {  
    perror( "listen" ); exit( EXIT_FAILURE );  
}
```

Attente active sur la socket

TCP client initialization example

```
struct sockaddr_in sock_host;
int sock;
sock = socket(AF_INET, SOCK_STREAM, 0);
memset(& sock_host, '\0', sizeof(sock_host));
sock_host.sin_family = AF_INET;
sock_host.sin_port = htons(port);
inet_aton("192.168.1.10", & sock_host.sin_addr);
connect(sock, (struct sockaddr *) & sock_host, sizeof(sock_host));
```

Déclaration des variables

Ouverture de la socket

Création de l'adresse de la machine distante

Connexion à la station distante 192.168.1.10

Socket programming with UDP

UDP: no “connection” between client and server:

- no handshaking before sending data
- sender explicitly attaches IP destination address and port # to each packet
- receiver extracts sender IP address and port# from received packet

UDP: transmitted data may be lost or received out-of-order

Application viewpoint:

- UDP provides *unreliable* transfer of groups of bytes (“datagrams”) between client and server processes

Socket programming with UDP



Lecture overview

- Principles of network applications
- **Web and HTTP**
- The Domain Name System DNS
- P2P applications
- socket programming with UDP and TCP



Web and HTTP

First, a quick review...

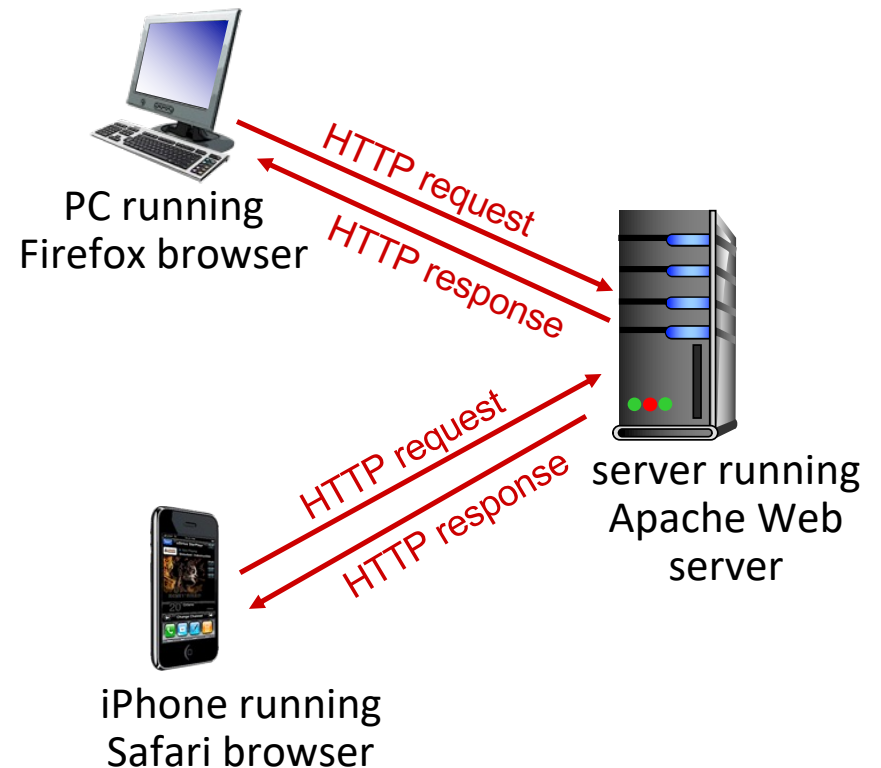
- web page consists of *objects*, each of which can be stored on different Web servers
- object can be HTML file, JPEG image, Java applet, audio file,...
- web page consists of *base HTML-file* which includes *several referenced objects*, *each* addressable by a *URL*, e.g.,

www.someschool.edu / someDept/pic.gif
host name path name

HTTP overview

HTTP: hypertext transfer protocol

- Web's application-layer protocol
- client/server model:
 - *client*: browser that requests, receives, (using HTTP protocol) and “displays” Web objects
 - *server*: Web server sends (using HTTP protocol) objects in response to requests



HTTP overview (continued)

HTTP uses TCP:

- client initiates TCP connection (creates socket) to server, port 80
- server accepts TCP connection from client
- HTTP messages (application-layer protocol messages) exchanged between browser (HTTP client) and Web server (HTTP server)
- TCP connection closed

HTTP is “stateless”

- server maintains *no* information about past client requests

aside

protocols that maintain “state” are complex!

- past history (state) must be maintained
- if server/client crashes, their views of “state” may be inconsistent, must be reconciled

HTTP connections: two types

Non-persistent HTTP

1. TCP connection opened
2. at most one object sent over TCP connection
3. TCP connection closed

downloading multiple objects
required multiple connections

Persistent HTTP

- TCP connection opened to a server
- multiple objects can be sent over *single* TCP connection between client, and that server
- TCP connection closed

Non-persistent HTTP: example

User enters URL: `www.someSchool.edu/someDepartment/home.index`
(containing text, references to 10 jpeg images)



time
↓

1a. HTTP client initiates TCP connection to HTTP server (process) at `www.someSchool.edu` on port 80

1b. HTTP server at host `www.someSchool.edu` waiting for TCP connection at port 80 “accepts” connection, notifying client

2. HTTP client sends HTTP *request message* (containing URL) into TCP connection socket. Message indicates that client wants object `someDepartment/home.index`

3. HTTP server receives request message, forms *response message* containing requested object, and sends message into its socket

Non-persistent HTTP: example (cont.)

User enters URL: `www.someSchool.edu/someDepartment/home.index`
(containing text, references to 10 jpeg images)



4. HTTP server closes TCP connection.

5. HTTP client receives response message containing html file, displays html. Parsing html file, finds 10 referenced jpeg objects

6. Steps 1-5 repeated for each of 10 jpeg objects

time

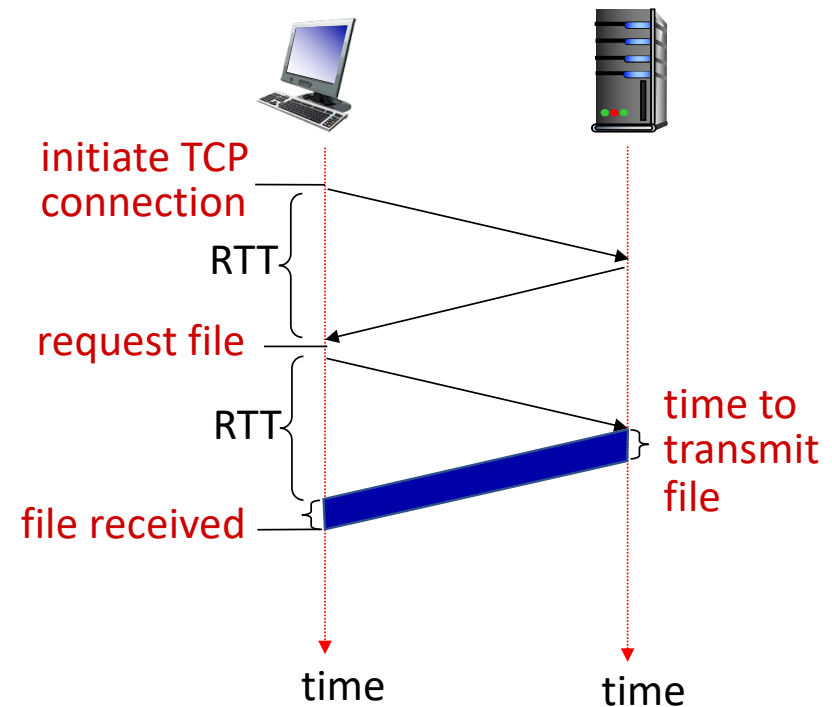


Non-persistent HTTP: response time

RTT (definition): time for a small packet to travel from client to server and back

HTTP response time (per object):

- one RTT to initiate TCP connection
- one RTT for HTTP request and first few bytes of HTTP response to return
- object/file transmission time



Non-persistent HTTP response time = $2RTT + \text{file transmission time}$

Persistent HTTP (HTTP 1.1)

Non-persistent HTTP issues:

- requires 2 RTTs per object
- OS overhead for *each* TCP connection
- browsers often open multiple parallel TCP connections to fetch referenced objects in parallel

Persistent HTTP (HTTP1.1):

- server leaves connection open after sending response
- subsequent HTTP messages between same client/server sent over open connection
- client sends requests as soon as it encounters a referenced object
- as little as one RTT for all the referenced objects (cutting response time in half)

HTTP request message

- two types of HTTP messages: *request, response*
- HTTP request message:
 - ASCII (human-readable format)

request line (GET, POST, HEAD commands) → GET /index.html HTTP/1.1\r\n

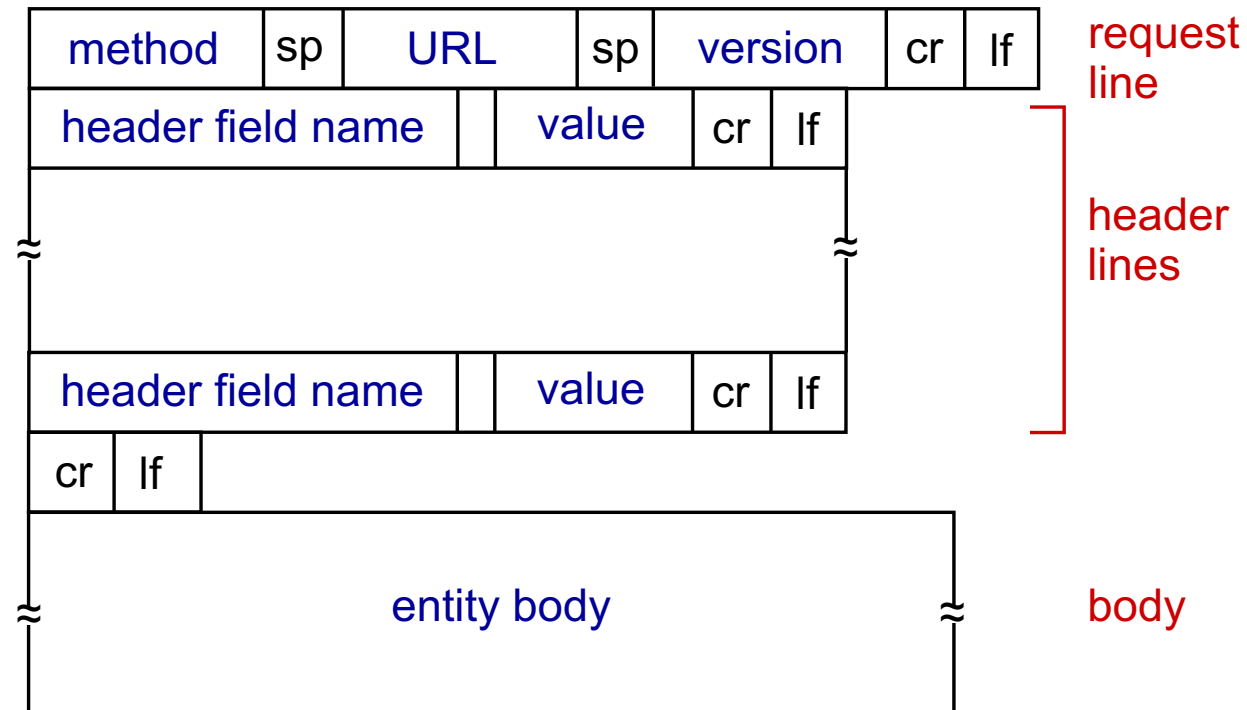
header lines → Host: www-net.cs.umass.edu\r\n
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:80.0) Gecko/20100101 Firefox/80.0 \r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Connection: keep-alive\r\n
\r\n

carriage return, line feed at start of line indicates end of header lines → \r\n

carriage return character
line-feed character

* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

HTTP request message: general format



Other HTTP request messages

POST method:

- web page often includes form input
- user input sent from client to server in entity body of HTTP POST request message

GET method (for sending data to server):

- include user data in URL field of HTTP GET request message (following a '?'):

`www.somesite.com/animalsearch?monkeys&banana`

HEAD method:

- requests headers (only) that would be returned *if* specified URL were requested with an HTTP GET method.

PUT method:

- uploads new file (object) to server
- completely replaces file that exists at specified URL with content in entity body of POST HTTP request message

HTTP response message

status line (protocol status code status phrase) → HTTP/1.1 200 OK

header lines {
Date: Tue, 08 Sep 2020 00:53:20 GMT
Server: Apache/2.4.6 (CentOS)
OpenSSL/1.0.2k-fips PHP/7.4.9
mod_perl/2.0.11 Perl/v5.16.3
Last-Modified: Tue, 01 Mar 2016 18:57:50 GMT
ETag: "a5b-52d015789ee9e"
Accept-Ranges: bytes
Content-Length: 2651
Content-Type: text/html; charset=UTF-8
\r\n

data, e.g., requested HTML file → data data data data data ...

* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

HTTP response status codes

- status code appears in 1st line in server-to-client response message.
- some sample codes:

200 OK

- request succeeded, requested object later in this message

301 Moved Permanently

- requested object moved, new location specified later in this message (in Location: field)

400 Bad Request

- request msg not understood by server

404 Not Found

- requested document not found on this server

505 HTTP Version Not Supported

Trying out HTTP (client side) for yourself

1. netcat to your favorite Web server:

% nc -c -v localhost 80

OR

% nc -v enseirb-matmeca.bordeaux-inp.fr 80

- opens TCP connection to port 80 (default HTTP server port) at enseirb-matmeca.bordeaux-inp.fr.
- anything typed in will be sent to port 80 at enseirb-matmeca.bordeaux-inp.fr

2. type in a GET HTTP request:

GET /index.html HTTP/1.1

Host: enseirb-matmeca.bordeaux-inp.fr

- by typing this in (hit carriage return twice), you send this minimal (but complete) GET request to HTTP server

3. look at response message sent by HTTP server!

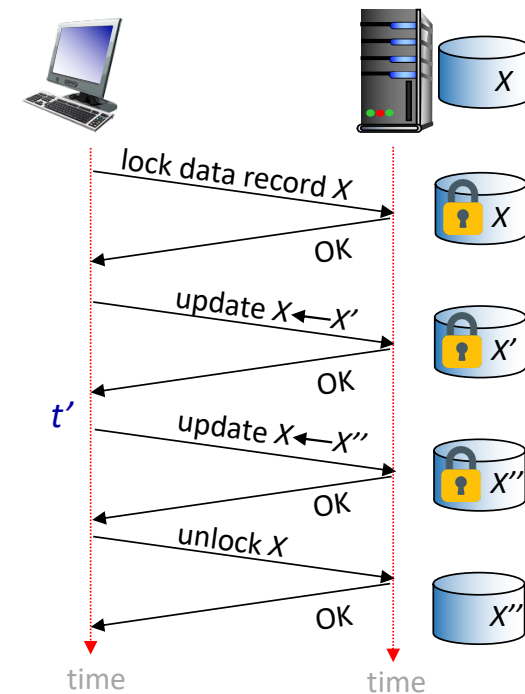
(or use Wireshark to look at captured HTTP request/response)

Maintaining user/server state: cookies

Recall: HTTP GET/response interaction is *stateless*

- no notion of multi-step exchanges of HTTP messages to complete a Web “transaction”
 - no need for client/server to track “state” of multi-step exchange
 - all HTTP requests are independent of each other
 - no need for client/server to “recover” from a partially-completed-but-never-completely-completed transaction

a **stateful protocol**: client makes two changes to X , or none at all



Q: what happens if network connection or client crashes at t' ?

Maintaining user/server state: cookies

Web sites and client browser use *cookies* to maintain some state between transactions

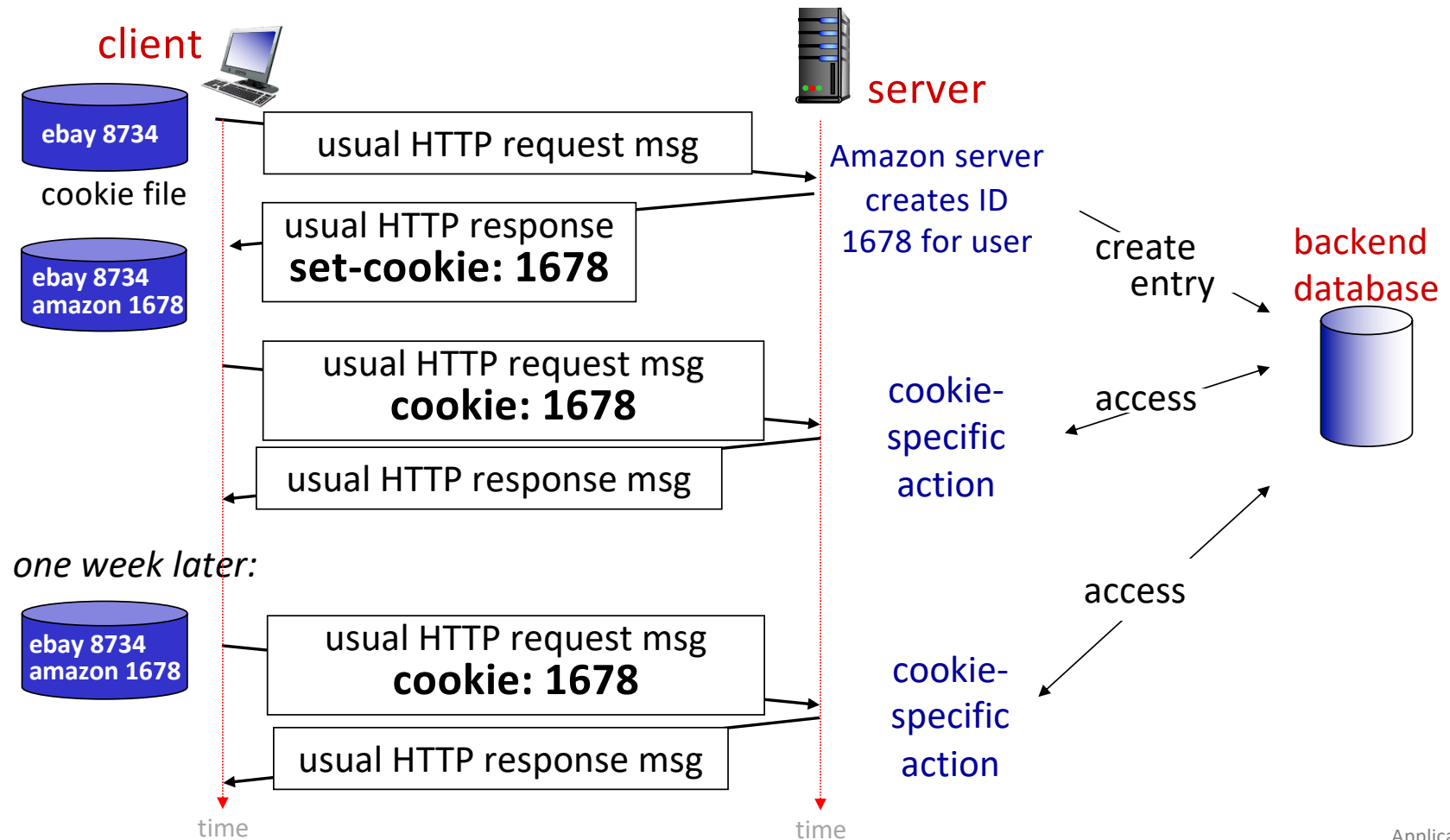
four components:

- 1) cookie header line of HTTP *response* message
- 2) cookie header line in next HTTP *request* message
- 3) cookie file kept on user's host, managed by user's browser
- 4) back-end database at Web site

Example:

- Susan uses browser on laptop, visits specific e-commerce site for first time
- when initial HTTP requests arrives at site, site creates:
 - unique ID (aka “cookie”)
 - entry in backend database for ID
- subsequent HTTP requests from Susan to this site will contain cookie ID value, allowing site to “identify” Susan

Maintaining user/server state: cookies



HTTP cookies: comments

What cookies can be used for:

- authorization
- shopping carts
- recommendations
- user session state (Web e-mail)

Challenge: How to keep state?

- *at protocol endpoints:* maintain state at sender/receiver over multiple transactions
- *in messages:* cookies in HTTP messages carry state

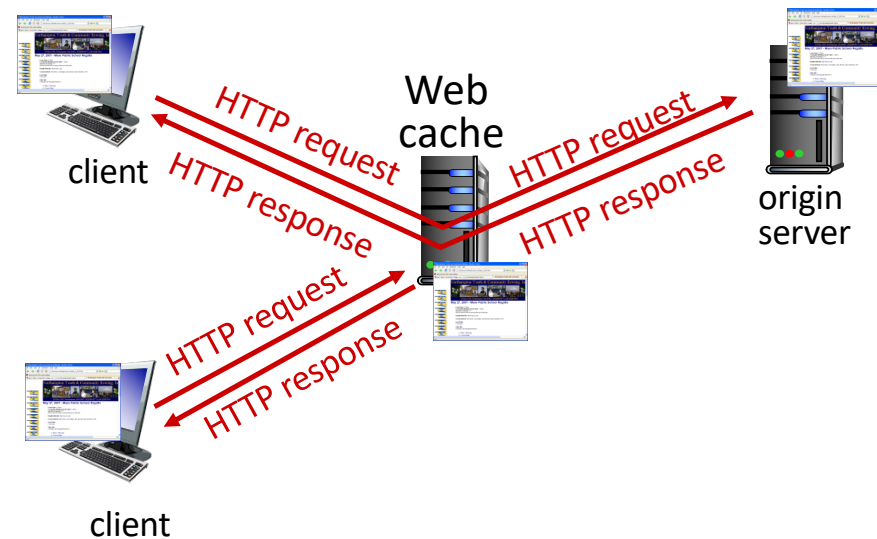
cookies and privacy: aside

- cookies permit sites to *learn* a lot about you on their site.
- third party persistent cookies (tracking cookies) allow common identity (cookie value) to be tracked across multiple web sites

Web caches

Goal: satisfy client requests without involving origin server

- user configures browser to point to a (local) *Web cache*
- browser sends all HTTP requests to cache
 - *if* object in cache: cache returns object to client
 - *else* cache requests object from origin server, caches received object, then returns object to client



Web caches (aka proxy servers)

- Web cache acts as both client and server
 - server for original requesting client
 - client to origin server
- server tells cache about object's allowable caching in response header:

```
Cache-Control: max-age=<seconds>
```

```
Cache-Control: no-cache
```

Why Web caching?

- reduce response time for client request
 - cache is closer to client
- reduce traffic on an institution's access link
- Internet is dense with caches
 - enables “poor” content providers to more effectively deliver content

Caching example

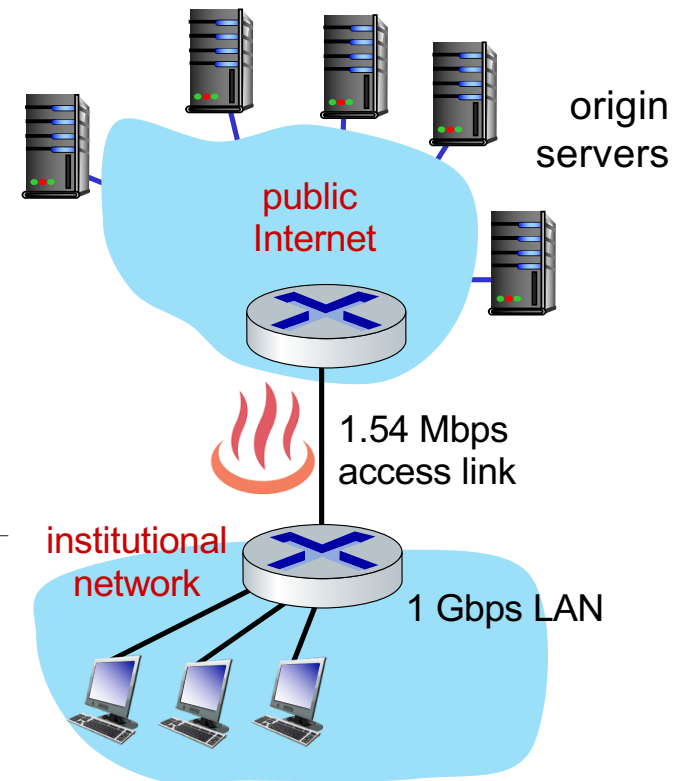
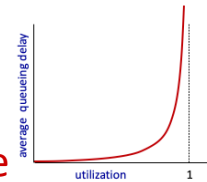
Scenario:

- access link rate: 1.54 Mbps
- RTT from institutional router to server: 2 sec
- web object size: 100K bits
- average request rate from browsers to origin servers: 15/sec
 - avg data rate to browsers: 1.50 Mbps

Performance:

- access link utilization = .97
- LAN utilization: .0015
- end-end delay = Internet delay +
access link delay + LAN delay
= 2 sec + minutes + usecs

problem: large
queueing delays
at high utilization!



Option 1: buy a faster access link

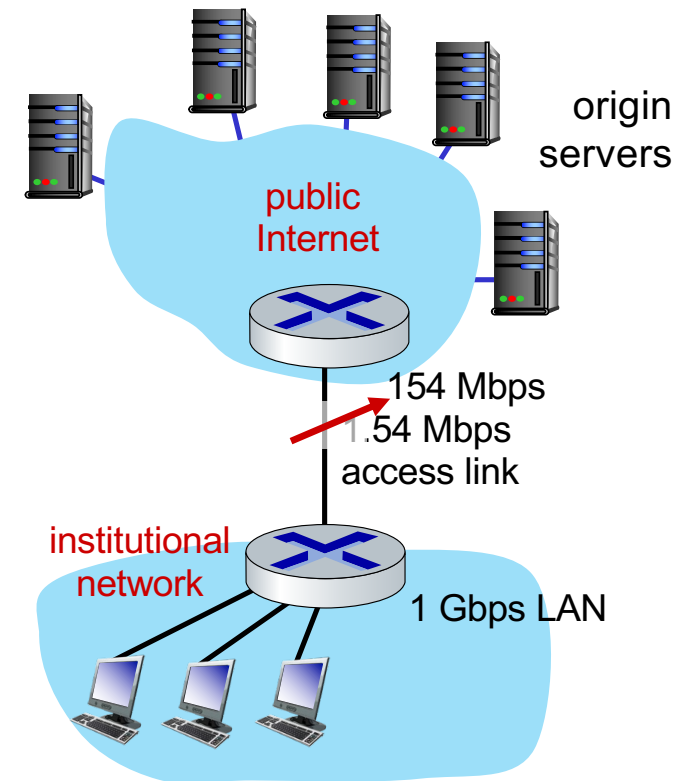
Scenario:

- access link rate: ~~1.54~~ 154 Mbps
- RTT from institutional router to server: 2 sec
- web object size: 100K bits
- average request rate from browsers to origin servers: 15/sec
 - avg data rate to browsers: 1.50 Mbps

Performance:

- access link utilization = ~~.97~~ .0097
- LAN utilization: .0015
- end-end delay = Internet delay +
access link delay + LAN delay
= 2 sec + ~~minutes~~ + usecs

Cost: faster access link (expensive!) → msecs



Option 2: install a web cache

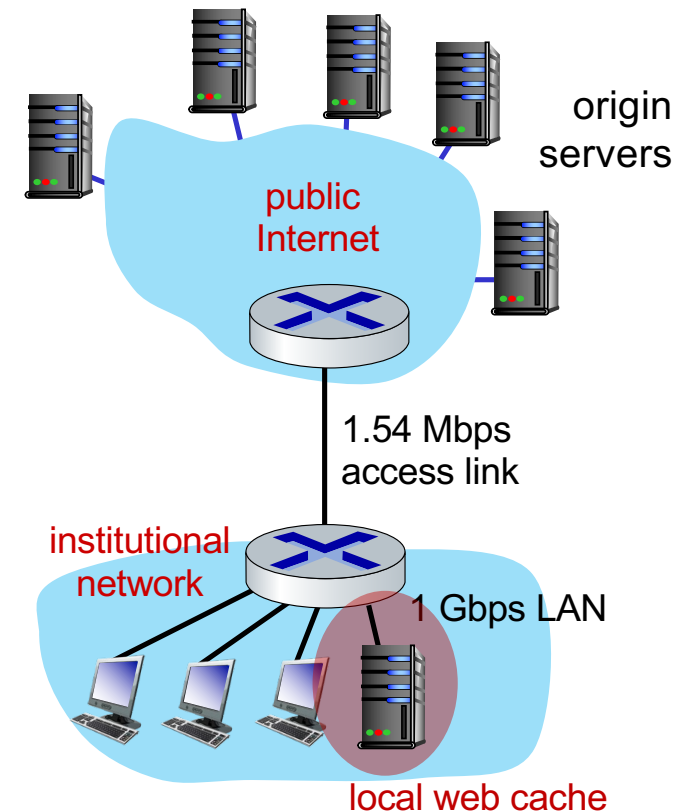
Scenario:

- access link rate: 1.54 Mbps
- RTT from institutional router to server: 2 sec
- web object size: 100K bits
- average request rate from browsers to origin servers: 15/sec
 - avg data rate to browsers: 1.50 Mbps

Cost: web cache (cheap!)

Performance:

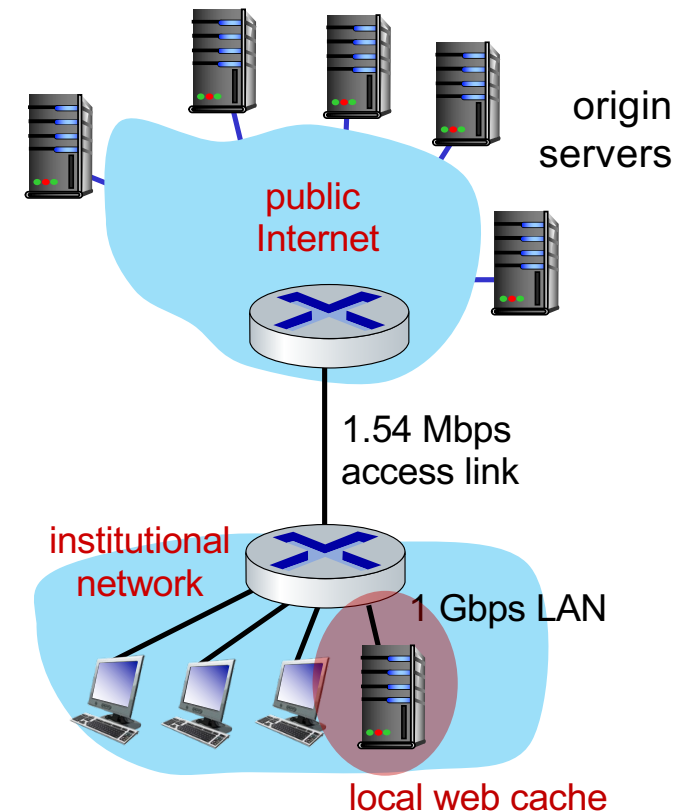
- LAN utilization: .?
 - access link utilization = ?
 - average end-end delay = ?
- How to compute link utilization, delay?*



Calculating access link utilization, end-end delay with cache:

suppose cache hit rate is 0.4:

- 40% requests served by cache, with low (msec) delay
- 60% requests satisfied at origin
 - rate to browsers over access link
 $= 0.6 * 1.50 \text{ Mbps} = .9 \text{ Mbps}$
 - access link utilization $= 0.9/1.54 = .58$ means low (msec) queueing delay at access link
- average end-end delay:
 $= 0.6 * (\text{delay from origin servers})$
 $+ 0.4 * (\text{delay when satisfied at cache})$
 $= 0.6 (2.01) + 0.4 (\sim \text{msecs}) = \sim 1.2 \text{ secs}$



lower average end-end delay than with 154 Mbps link (and cheaper too!)

Conditional GET

Goal: don't send object if cache has up-to-date cached version

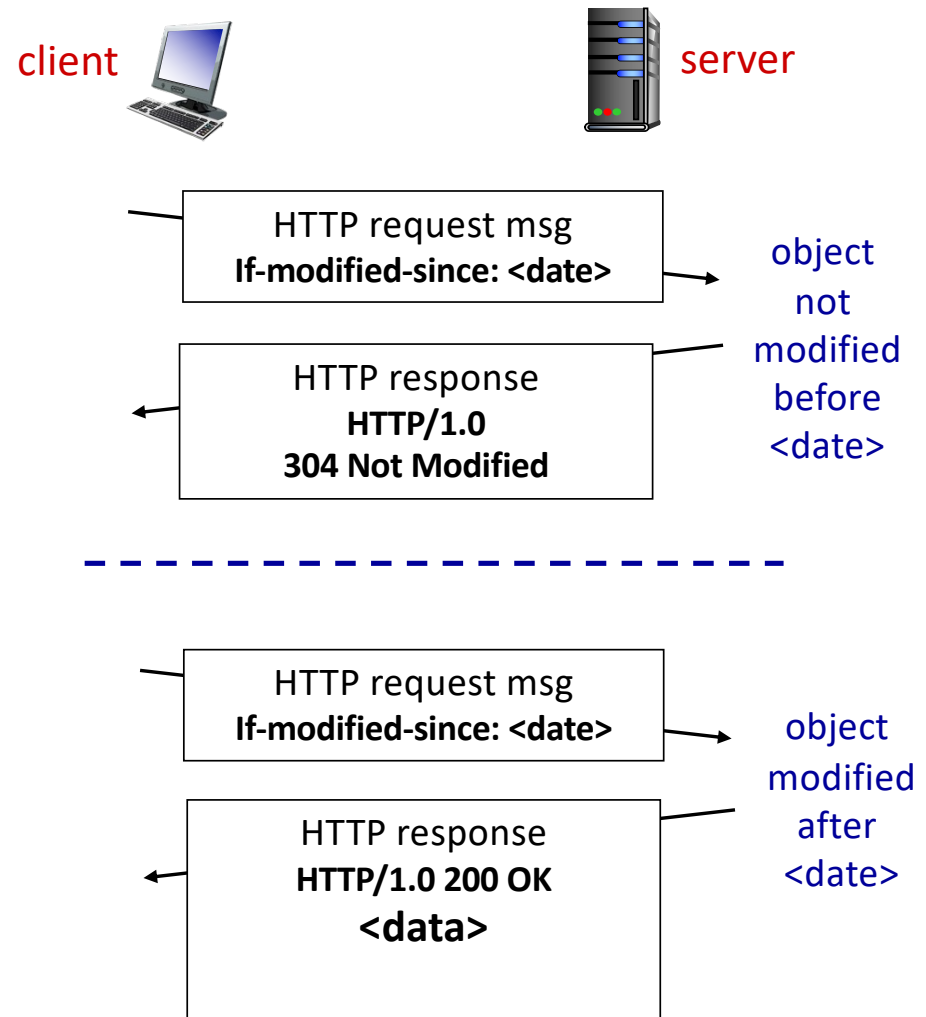
- no object transmission delay (or use of network resources)

- *client*: specify date of cached copy in HTTP request

If-modified-since: <date>

- *server*: response contains no object if cached copy is up-to-date:

HTTP/1.0 304 Not Modified



HTTP/2

Key goal: decreased delay in multi-object HTTP requests

HTTP1.1: introduced **multiple, pipelined GETs** over single TCP connection

- server responds *in-order* (FCFS: first-come-first-served scheduling) to GET requests
- with FCFS, small object may have to wait for transmission (**head-of-line (HOL) blocking**) behind large object(s)
- loss recovery (retransmitting lost TCP segments) stalls object transmission

HTTP/2

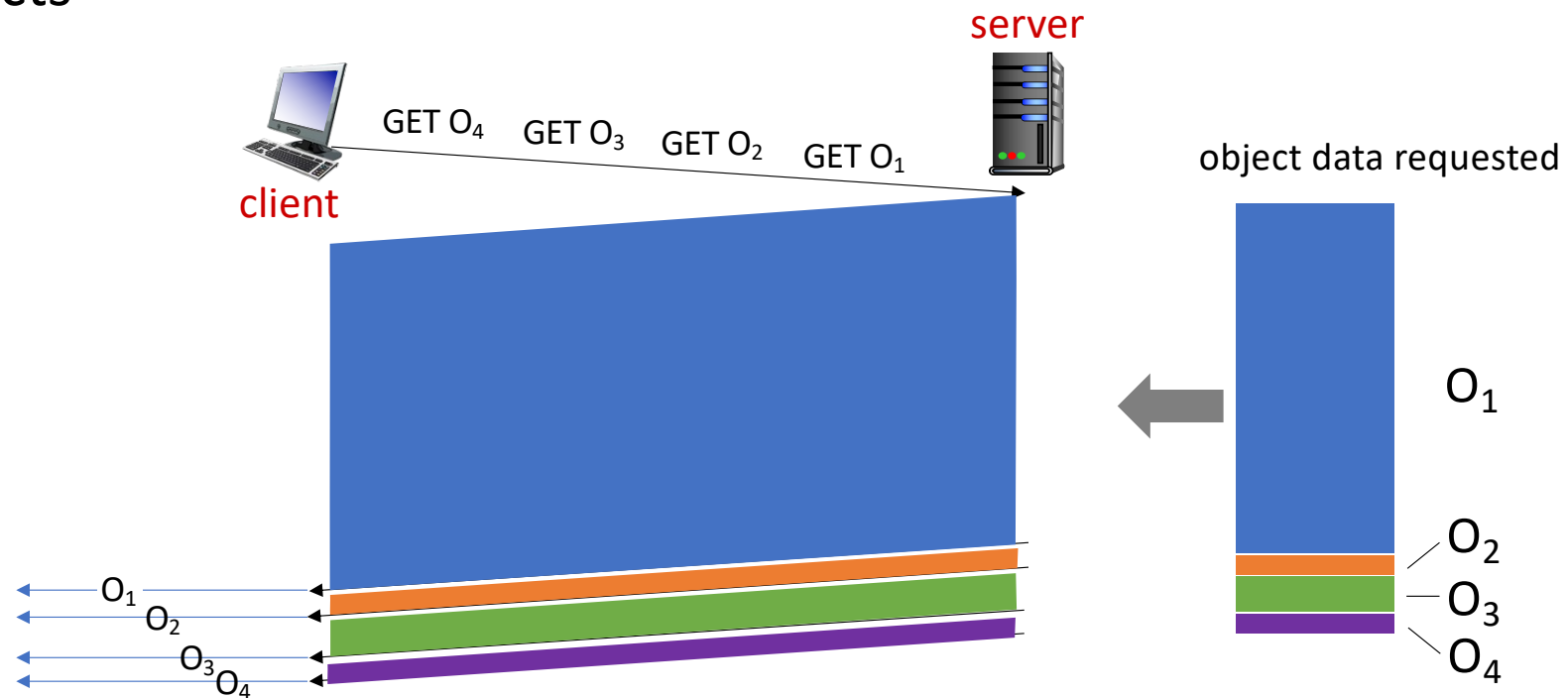
Key goal: decreased delay in multi-object HTTP requests

HTTP/2: [RFC 7540, 2015] increased flexibility at *server* in sending objects to client:

- methods, status codes, most header fields unchanged from HTTP 1.1
- transmission order of requested objects based on client-specified object priority (not necessarily FCFS)
- *push* unrequested objects to client
- divide objects into frames, schedule frames to mitigate HOL blocking

HTTP/2: mitigating HOL blocking

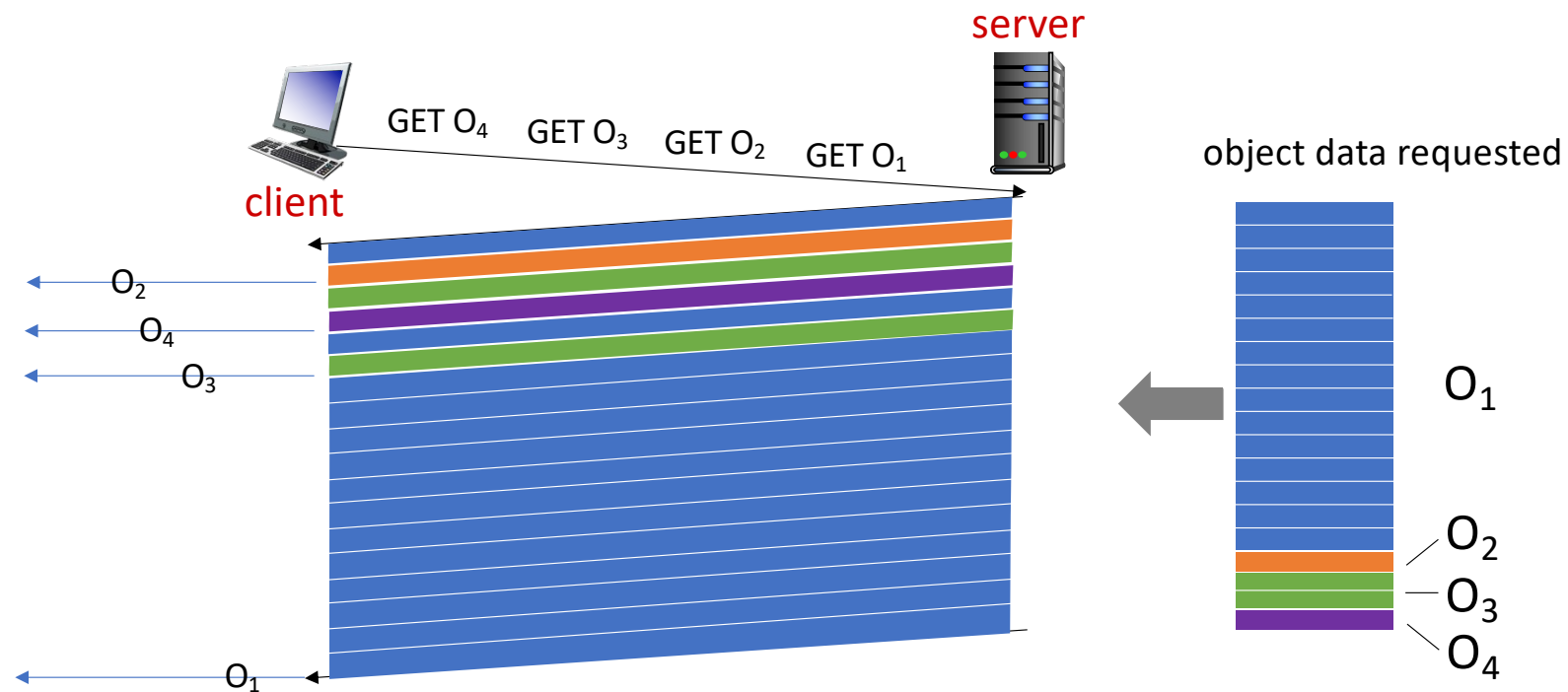
HTTP 1.1: client requests 1 large object (e.g., video file) and 3 smaller objects



objects delivered in order requested: O_2 , O_3 , O_4 wait behind O_1

HTTP/2: mitigating HOL blocking

HTTP/2: objects divided into frames, frame transmission interleaved



`O2`, `O3`, `O4` delivered quickly, `O1` slightly delayed

HTTP/2 to HTTP/3

HTTP/2 over single TCP connection means:

- recovery from packet loss still stalls all object transmissions
 - as in HTTP 1.1, browsers have incentive to open multiple parallel TCP connections to reduce stalling, increase overall throughput
- no security over vanilla TCP connection
- **HTTP/3**: adds security, per object error- and congestion-control (more pipelining) over UDP
 - more on HTTP/3 in transport layer (not seen here)

Lecture overview

- Principles of network applications
- Web and HTTP
- **The Domain Name System
DNS**
- P2P applications
- socket programming with UDP and TCP



DNS: Domain Name System

people: many identifiers:

- SSN, name, passport #

Internet hosts, routers:

- IP address (32 bit) - used for addressing datagrams
- “name”, e.g., cs.umass.edu - used by humans

Q: how to map between IP address and name, and vice versa ?

Domain Name System (DNS):

- *distributed database* implemented in hierarchy of many *name servers*
- *application-layer protocol*: hosts, DNS servers communicate to *resolve* names (address/name translation)
 - *note*: core Internet function, implemented as application-layer protocol
 - complexity at network’s “edge”

DNS: services, structure

DNS services:

- hostname-to-IP-address translation
- host aliasing
 - canonical, alias names
- mail server aliasing
- load distribution
 - replicated Web servers: many IP addresses correspond to one name

Q: Why not centralize DNS?

- single point of failure
- traffic volume
- distant centralized database
- maintenance

A: doesn't scale!

- Comcast DNS servers alone: 600B DNS queries/day
- Akamai DNS servers alone: 2.2T DNS queries/day

Thinking about the DNS

humongous distributed database:

- ~ billion records, each simple

handles many *trillions* of queries/day:

- *many* more reads than writes
- *performance matters*: almost every Internet transaction interacts with DNS - msec count!

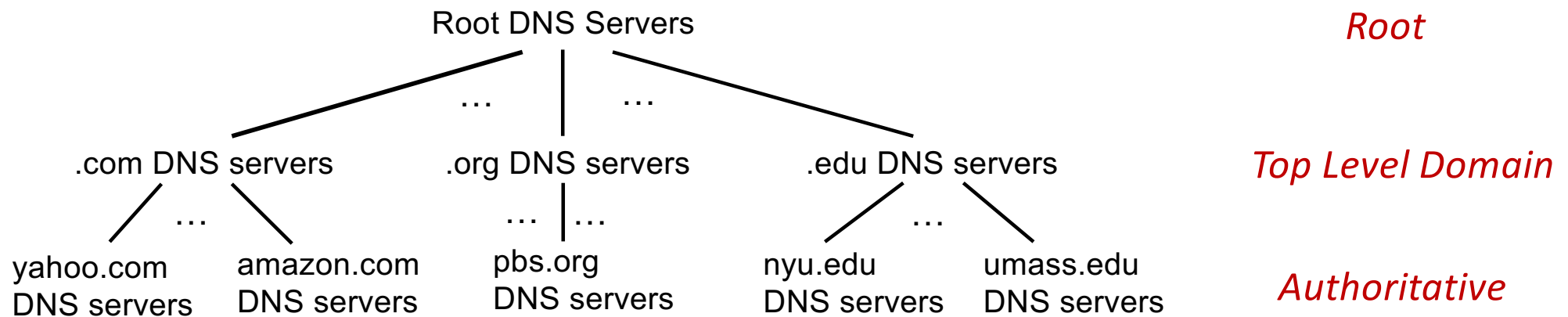
organizationally, physically decentralized:

- millions of different organizations responsible for their records

“bulletproof”: reliability, security



DNS: a distributed, hierarchical database

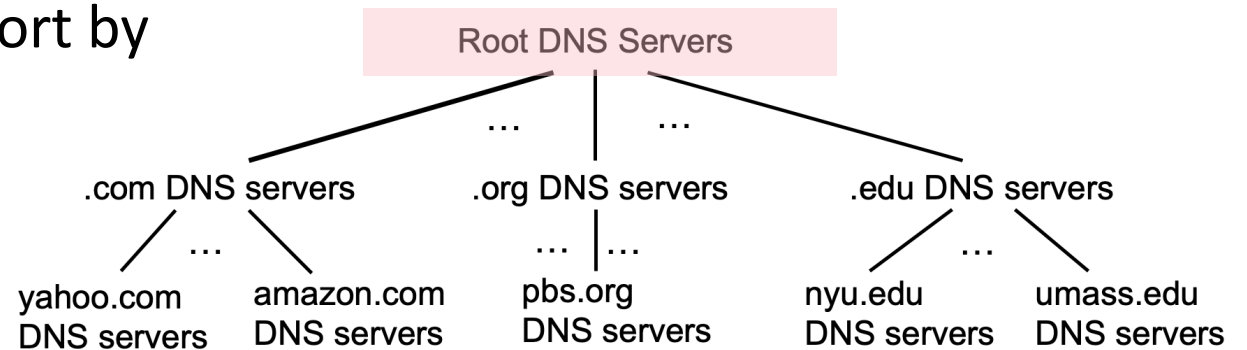


Client wants IP address for www.amazon.com; 1st approximation:

- client queries root server to find .com DNS server
- client queries .com DNS server to get amazon.com DNS server
- client queries amazon.com DNS server to get IP address for www.amazon.com

DNS: root name servers

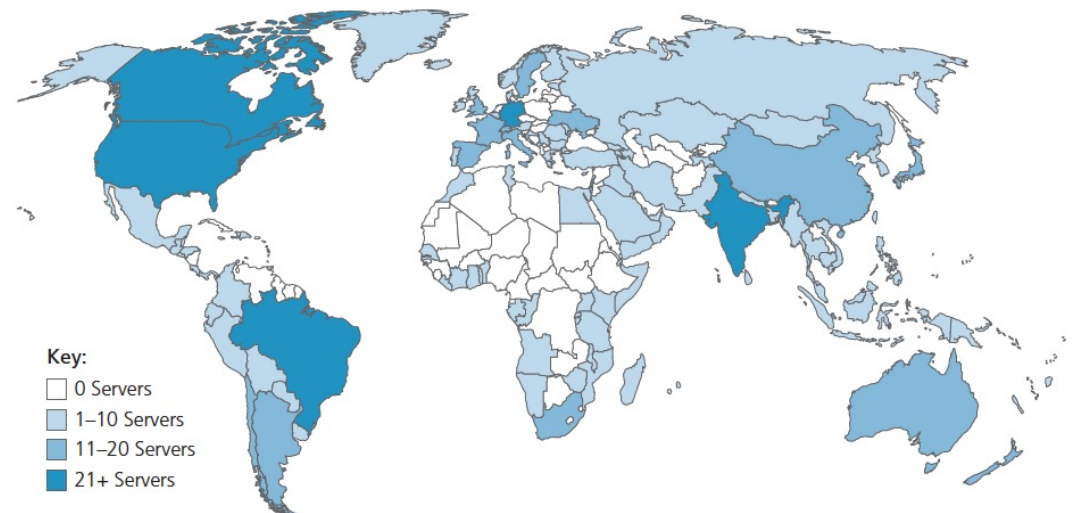
- official, contact-of-last-resort by name servers that can not resolve name



DNS: root name servers

- official, contact-of-last-resort by name servers that can not resolve name
- *incredibly important* Internet function
 - Internet couldn't function without it!
 - DNSSEC – provides security (authentication, message integrity)
- ICANN (Internet Corporation for Assigned Names and Numbers) manages root DNS domain

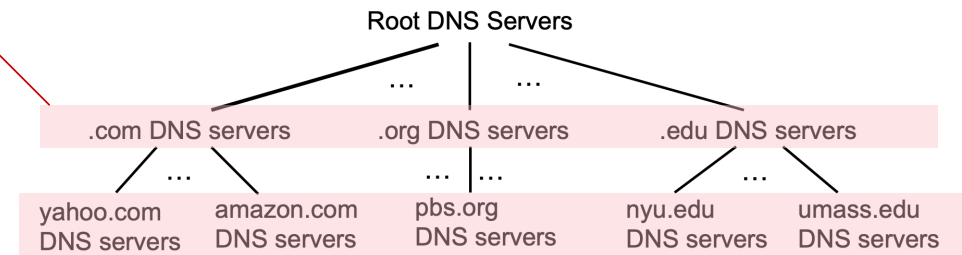
13 logical root name “servers”
worldwide each “server” replicated
many times (~200 servers in US)



Top-Level Domain, and authoritative servers

Top-Level Domain (TLD) servers:

- responsible for .com, .org, .net, .edu, .aero, .jobs, .museums, and all top-level country domains, e.g.: .cn, .uk, .fr, .ca, .jp
- Network Solutions: authoritative registry for .com, .net TLD
- Educause: .edu TLD



authoritative DNS servers:

- organization's own DNS server(s), providing authoritative hostname to IP mappings for organization's named hosts
- can be maintained by organization or service provider

Local DNS name servers

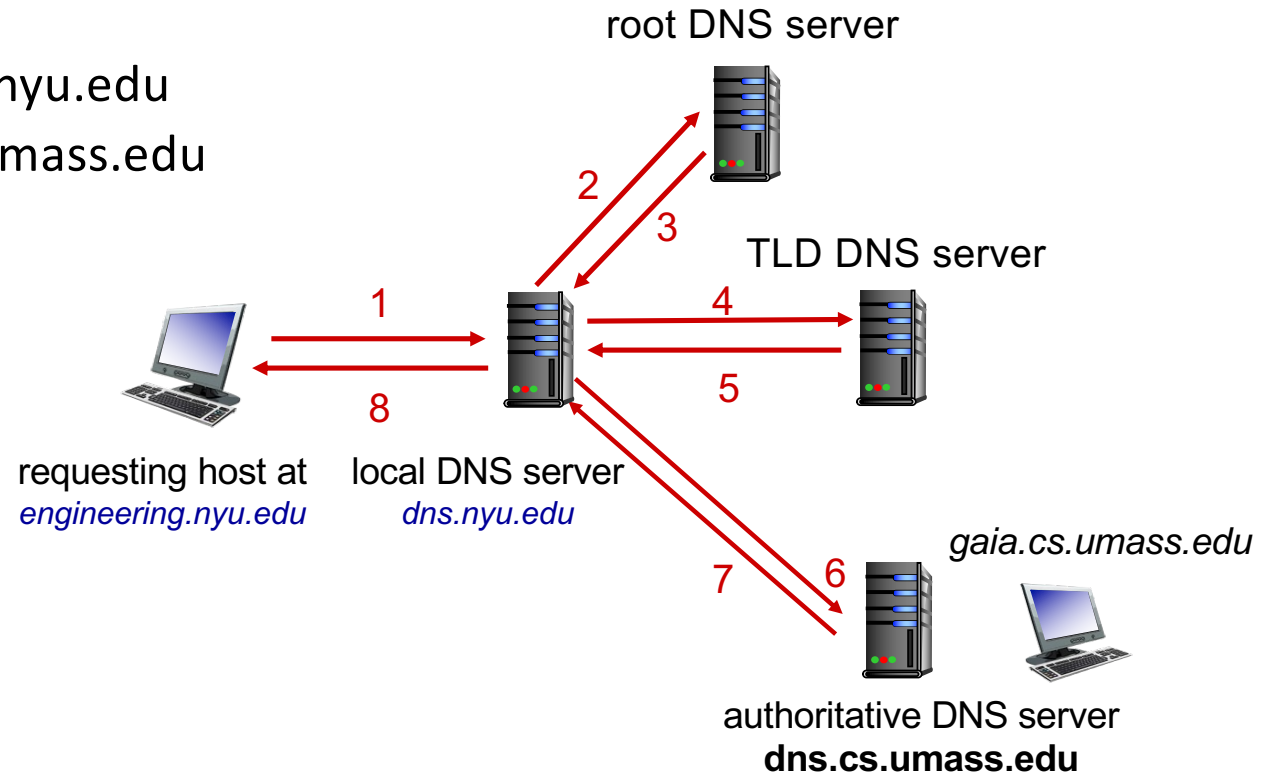
- when host makes DNS query, it is sent to its *local* DNS server
 - Local DNS server returns reply, answering:
 - from its local cache of recent name-to-address translation pairs (possibly out of date!)
 - forwarding request into DNS hierarchy for resolution
 - each ISP has local DNS name server; to find yours:
 - MacOS: `% scutil --dns`
 - Windows: `>ipconfig /all`
- local DNS server doesn't strictly belong to hierarchy

DNS name resolution: iterated query

Example: host at `engineering.nyu.edu` wants IP address for `gaia.cs.umass.edu`

Iterated query:

- contacted server replies with name of server to contact
- “I don’t know this name, but ask this server”

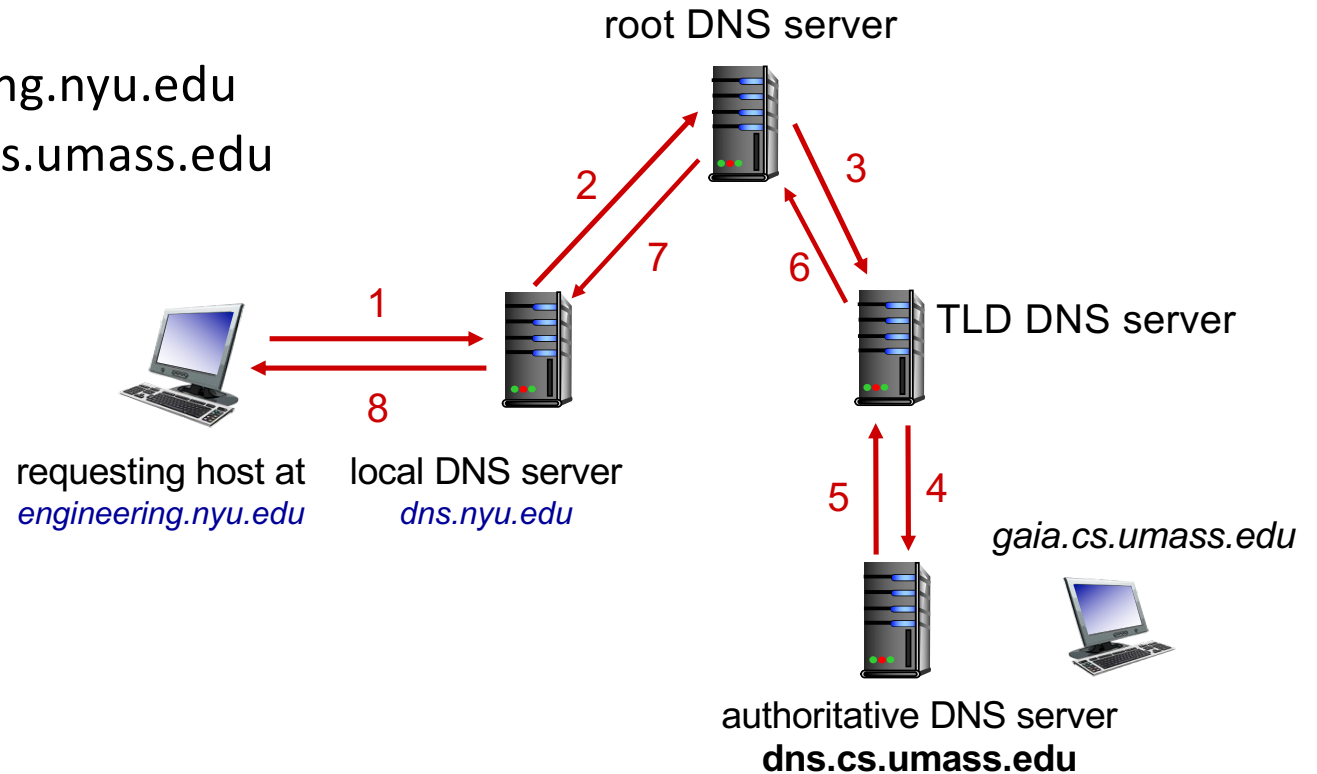


DNS name resolution: recursive query

Example: host at `engineering.nyu.edu` wants IP address for `gaia.cs.umass.edu`

Recursive query:

- puts burden of name resolution on contacted name server
- heavy load at upper levels of hierarchy?



Caching DNS Information

- once (any) name server learns mapping, it *caches* mapping, and *immediately* returns a cached mapping in response to a query
 - caching improves response time
 - cache entries timeout (disappear) after some time (TTL)
 - TLD servers typically cached in local name servers
- cached entries may be *out-of-date*
 - if named host changes IP address, may not be known Internet-wide until all TTLs expire!
 - *best-effort name-to-address translation!*

DNS records

DNS: distributed database storing resource records (RR)

RR format: (name, value, type, ttl)

type=A

- name is hostname
- value is IP address

type=NS

- name is domain (e.g., foo.com)
- value is hostname of authoritative name server for this domain

type=CNAME

- name is alias name for some “canonical” (the real) name
- www.ibm.com is really servereast.backup2.ibm.com
- value is canonical name

type=MX

- value is name of SMTP mail server associated with name

DNS protocol messages

DNS *query* and *reply* messages, both have same *format*:

message header:

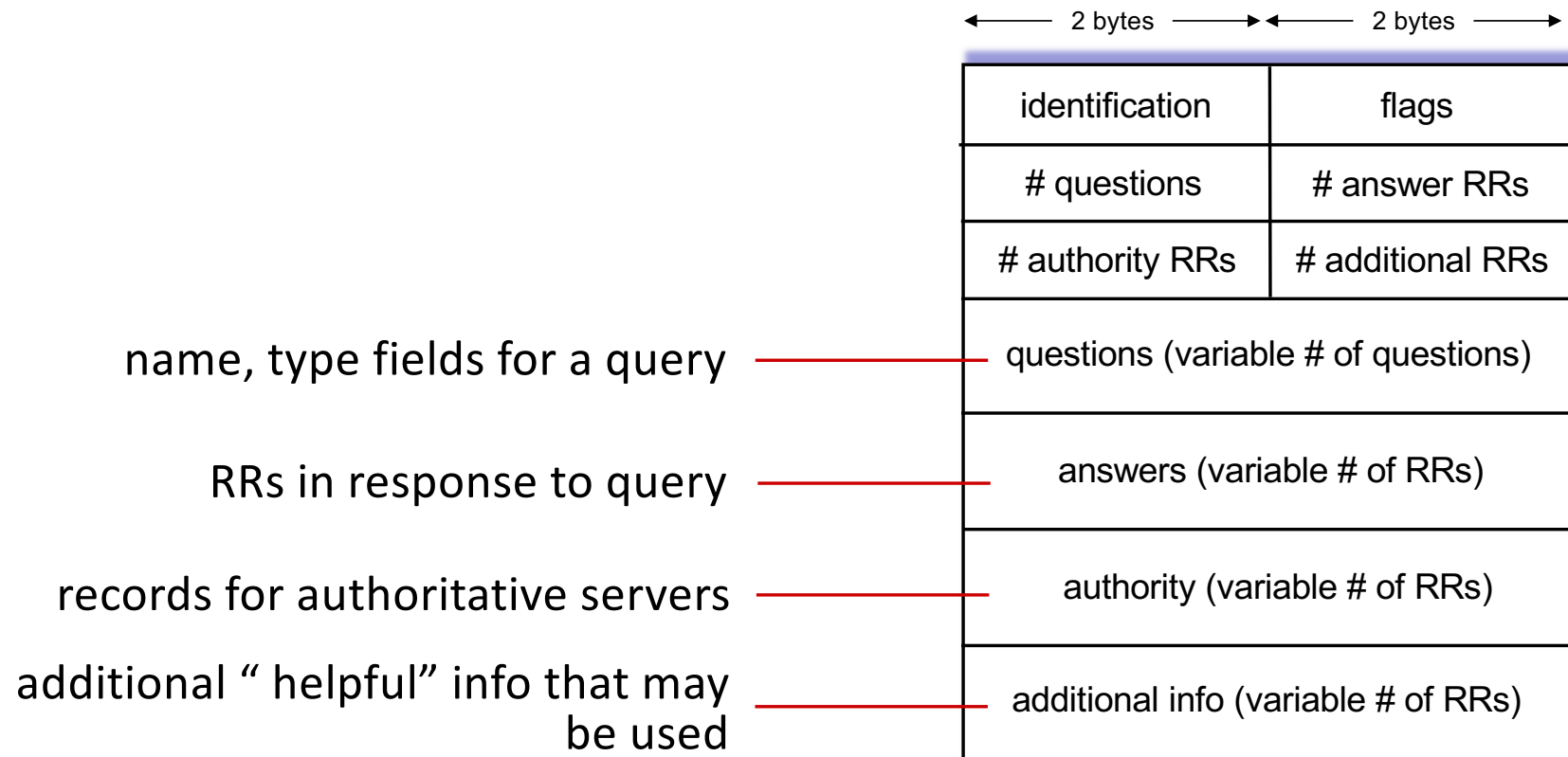
- **identification**: 16 bit # for query, reply to query uses same #
- **flags**:
 - query or reply
 - recursion desired
 - recursion available
 - reply is authoritative

← 2 bytes → ← 2 bytes →

identification	flags
# questions	# answer RRs
# authority RRs	# additional RRs
questions (variable # of questions)	
answers (variable # of RRs)	
authority (variable # of RRs)	
additional info (variable # of RRs)	

DNS protocol messages

DNS *query* and *reply* messages, both have same *format*:



Getting your info into the DNS

example: new startup “Network Utopia”

- register name networkutopia.com at *DNS registrar* (e.g., Network Solutions)
 - provide names, IP addresses of authoritative name server (primary and secondary)
 - registrar inserts NS, A RRs into .com TLD server:
`(networkutopia.com, dns1.networkutopia.com, NS)`
`(dns1.networkutopia.com, 212.212.212.1, A)`
- create authoritative server locally with IP address `212.212.212.1`
 - type A record for `www.networkutopia.com`
 - type MX record for `networkutopia.com`

DNS security

DDoS attacks

- bombard root servers with traffic
 - not successful to date
 - traffic filtering
 - local DNS servers cache IPs of TLD servers, allowing root server bypass
- bombard TLD servers
 - potentially more dangerous

Spoofing attacks

- intercept DNS queries, returning bogus replies
 - DNS cache poisoning
 - RFC 4033: DNSSEC authentication services

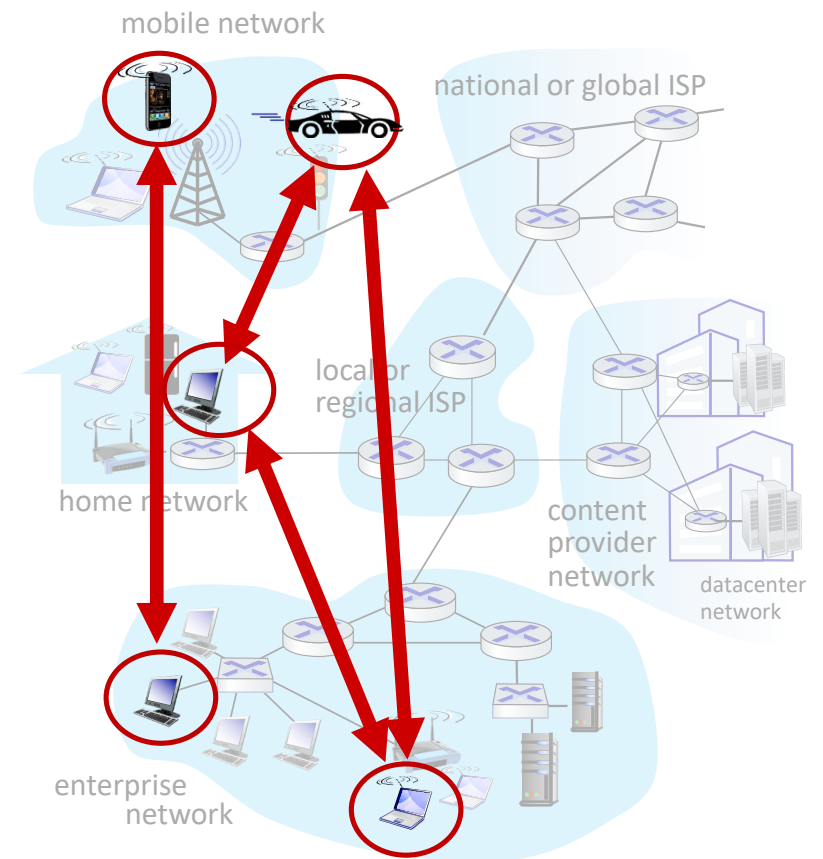
Lecture overview

- Principles of network applications
- Web and HTTP
- The Domain Name System DNS
- **P2P applications**
- socket programming with UDP and TCP



Peer-to-peer (P2P) architecture

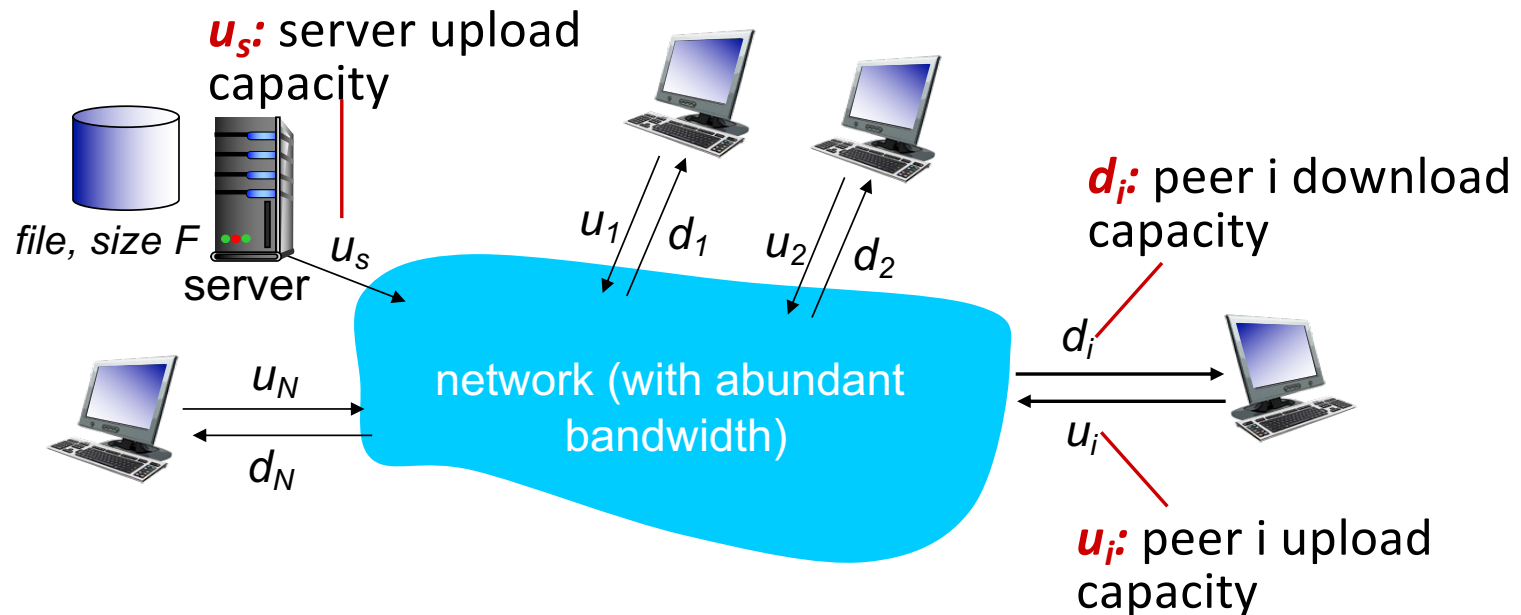
- *no* always-on server
- arbitrary end systems directly communicate
- peers request service from other peers, provide service in return to other peers
 - *self scalability* – new peers bring new service capacity, and new service demands
- peers are intermittently connected and change IP addresses
 - complex management
- examples: P2P file sharing (BitTorrent), streaming (KanKan), VoIP (Skype)



File distribution: client-server vs P2P

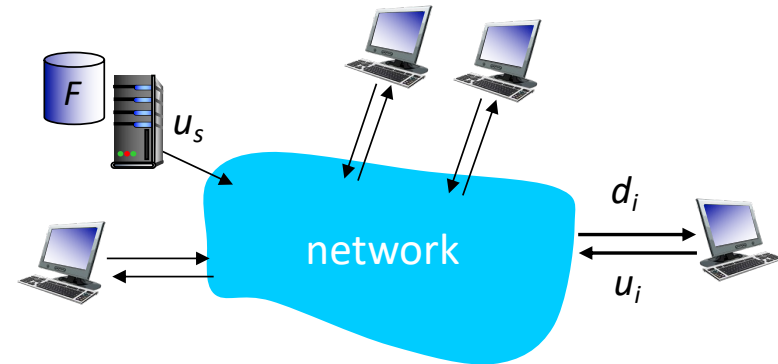
Q: how long to distribute file (size F) from one server to N peers?

- peer upload/download capacity is limited resource



File distribution time: client-server

- **server transmission:** must sequentially send (upload) N file copies:
 - time to send one copy: F/u_s
 - time to send N copies: NF/u_s
- **client:** each client must download file copy
 - d_{min} = min client download rate
 - min client download time: F/d_{min}



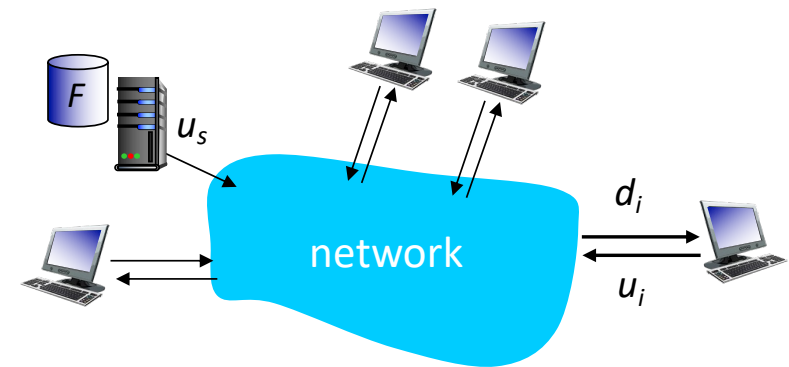
*time to distribute F
to N clients using
client-server approach*

$$D_{c-s} \geq \max\{NF/u_s, F/d_{min}\}$$

increases linearly in N

File distribution time: P2P

- **server transmission:** must upload at least one copy:
 - time to send one copy: F/u_s
- **client:** each client must download file copy
 - min client download time: F/d_{min}
- **clients:** as aggregate must download NF bits
 - max upload rate (limiting max download rate) is $u_s + \sum u_i$



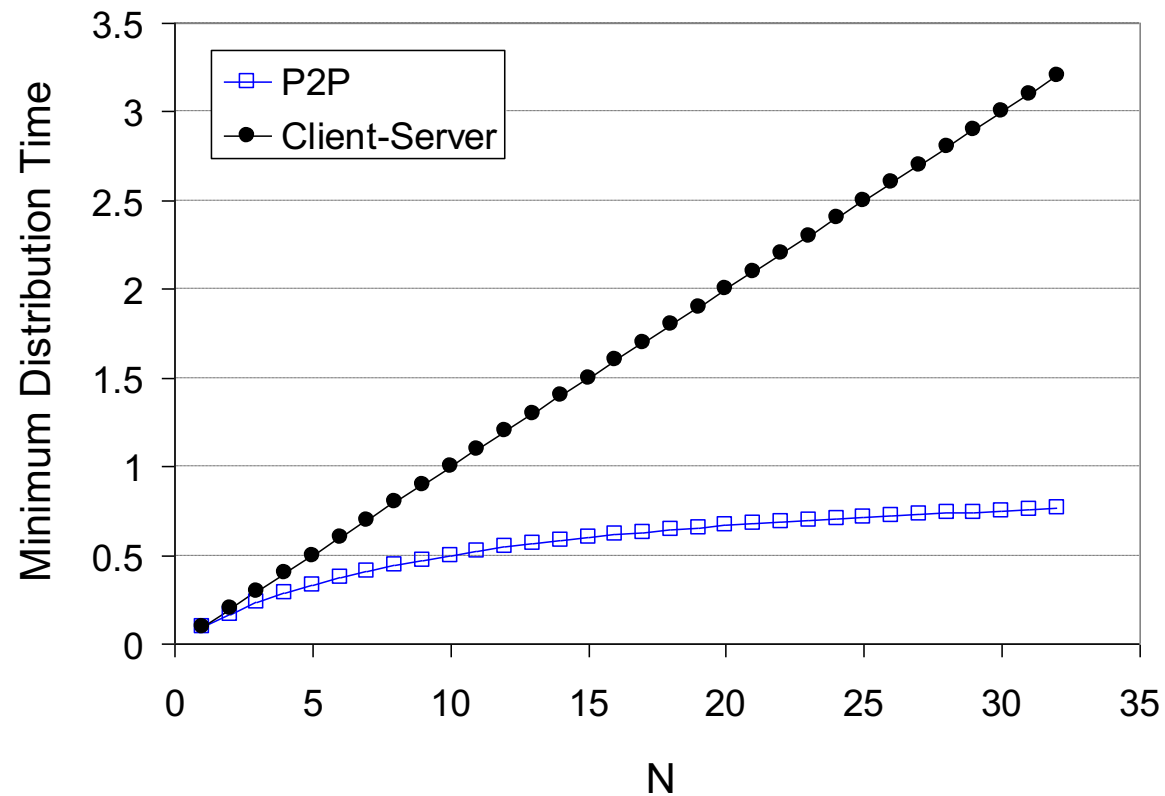
time to distribute F
to N clients using
P2P approach

$$D_{P2P} \geq \max\{F/u_s, F/d_{min}, NF/(u_s + \sum u_i)\}$$

increases linearly in N ...
... but so does this, as each peer brings service capacity

Client-server vs. P2P: example

client upload rate = u , $F/u = 1$ hour, $u_s = 10u$, $d_{min} \geq u_s$

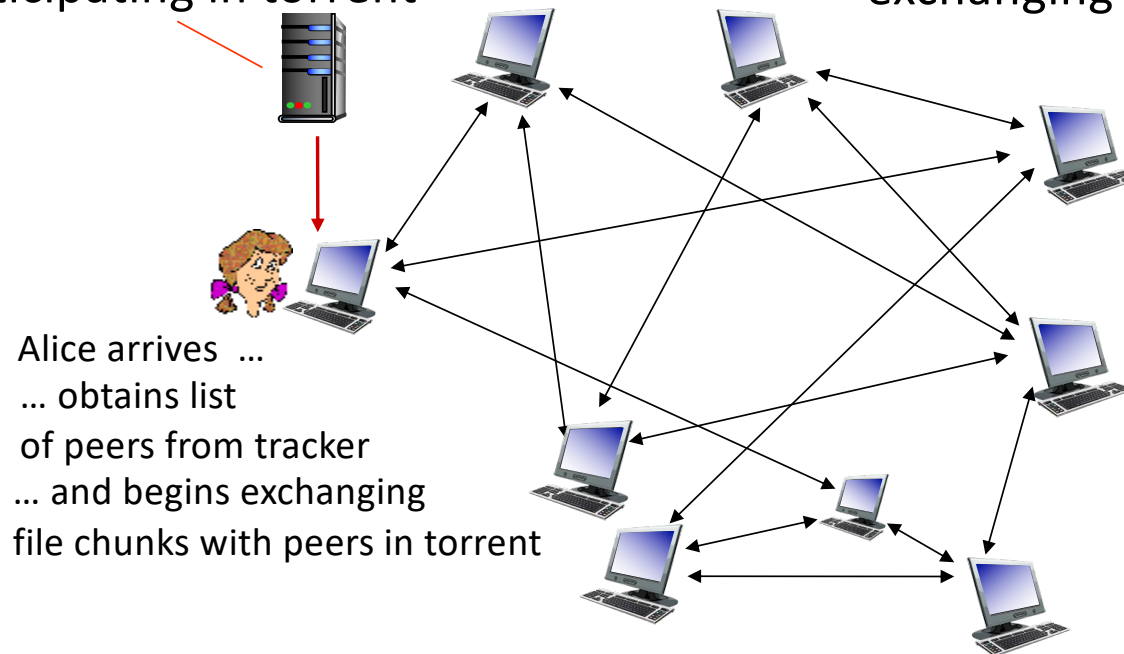


P2P file distribution: BitTorrent

- file divided into 256Kb chunks
- peers in torrent send/receive file chunks

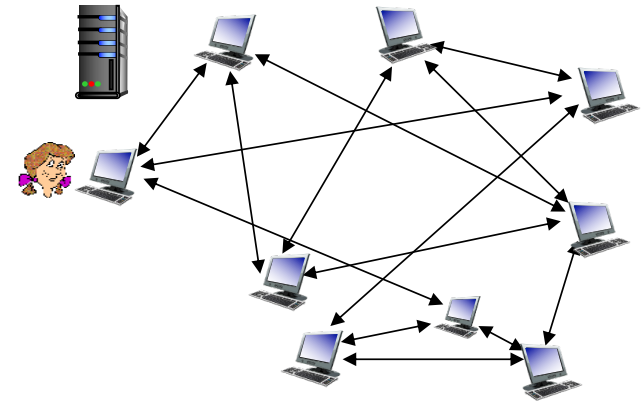
tracker: tracks peers participating in torrent

torrent: group of peers exchanging chunks of a file



P2P file distribution: BitTorrent

- peer joining torrent:
 - has no chunks, but will accumulate them over time from other peers
 - registers with tracker to get list of peers, connects to subset of peers (“neighbors”)
- while downloading, peer uploads chunks to other peers
- peer may change peers with whom it exchanges chunks
- *churn*: peers may come and go
- once peer has entire file, it may (selfishly) leave or (altruistically) remain in torrent



BitTorrent: requesting, sending file chunks

Requesting chunks:

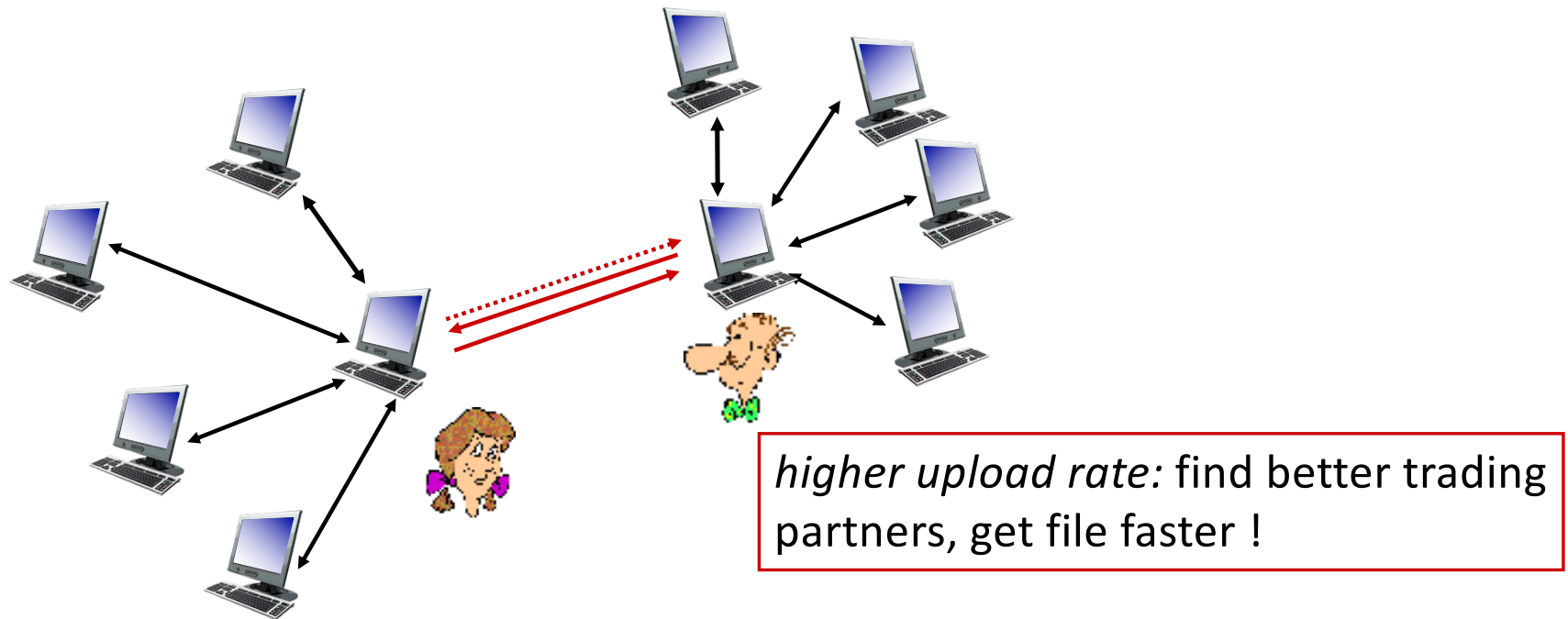
- at any given time, different peers have different subsets of file chunks
- periodically, Alice asks each peer for list of chunks that they have
- Alice requests missing chunks from peers, rarest first

Sending chunks: tit-for-tat

- Alice sends chunks to those four peers currently sending her chunks *at highest rate*
 - other peers are choked by Alice (do not receive chunks from her)
 - re-evaluate top 4 every 10 secs
- every 30 secs: randomly select another peer, starts sending chunks
 - “optimistically unchoke” this peer
 - newly chosen peer may join top 4

BitTorrent: tit-for-tat

- (1) Alice “optimistically unchokes” Bob
- (2) Alice becomes one of Bob’s top-four providers; Bob reciprocates
- (3) Bob becomes one of Alice’s top-four providers



Lecture summary

our study of network application layer is now complete!

- application architectures
 - client-server
 - P2P
- application service requirements:
 - reliability, bandwidth, delay
- Internet transport service model
 - connection-oriented, reliable: TCP
 - unreliable, datagrams: UDP
- specific protocols:
 - HTTP
 - SMTP, IMAP*
 - DNS
 - P2P: BitTorrent
- video streaming, CDNs
- socket programming:
TCP, UDP sockets

Lecture summary

Most importantly: learned about *protocols*!

- typical request/reply message exchange:
 - client requests info or service
 - server responds with data, status code
- message formats:
 - *headers*: fields giving info about data
 - *data*: info(payload) being communicated

important themes:

- centralized vs. decentralized
- stateless vs. stateful
- scalability
- reliable vs. unreliable message transfer

The End